`

SKRIPSI

IMPLEMENTASI METODE FRAME DIFFERENCE UNTUK MENDETEKSI KECEPATAN KENDARAAN BERMOTOR DI UNIVERSITAS RIAU

Oleh:

Muhamad Indra Setiabudi
NIM: 1407114645

PROGRAM STUDI TEKNIK INFORMATIKA S1
JURUSAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS RIAU
2021

SKRIPSI

IMPLEMENTASI METODE FRAME DIFFERENCE UNTUK MENDETEKSI KECEPATAN KENDARAAN BERMOTOR DI UNIVERSITAS RIAU

Diajukan untuk Memenuhi
Persyaratan Meraih Gelar Sarjana Teknik
Teknik Informatika Fakultas Teknik Universitas Riau

Oleh:

Muhamad Indra Setiabudi
NIM : 1407114645

PROGRAM STUDI TEKNIK INFORMATIKA S1
JURUSAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS RIAU
2021

HALAMAN PENGESAHAN

Skripsi dengan judul **"Implementasi Metode Frame Difference Untuk Mendeteksi Kecepatan Kendaraan Bermotor Di Universitas Riau"**

Yang dipersiapkan dan disusun oleh
Muhamad Indra Setiabudi
NIM: 1407114645
Program Studi Teknik Informatika S1, Fakultas Teknik, Universitas Riau

Telah berhasil dipertahankan di hadapan Tim Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik pada
Tanggal 15 Juli 2021

Menyetujui,
Pembimbing Skripsi

Dr. Feri Candra, ST.,MT
NIP. 19740428 200212 1 003

Mengetahui,
Koordinator Program Studi Teknik Informatika S1
Fakultas Teknik Universitas Riau

Dr. Feri Candra, ST., MT
NIP. 19740428 200212 1 003

HALAMAN PERNYATAAN ORISINALITAS

Dengan ini menyatakan bahwa Skripsi dengan judul: **"Implementasi Metode Frame Difference Untuk Mendeteksi Kecepatan Kendaraan Bermotor Di Universitas Riau"** tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Pekanbaru, 16 Agustus 2021

Muhamad Indra Setiabudi

PRAKATA

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya, penulis dapat menyelesaikan skripsi ini. Penulisan skripsi ini dilakukan dalam rangka memenuhi salah satu syarat mencapai gelar Sarjana Teknik Jurusan Teknik Informatika pada Fakultas Teknik Universitas Riau. Penulis menyadari bahwa, tanpa bantuan dan bimbingan dari berbagai pihak sangatlah sulit bagi penulis untuk menyelesaikan skripsi ini. Oleh karena itu, penulis mengucapkan terima kasih kepada:.

1. Orang tua
2. Dosen Pembimbing I Bapak Dr. Feri Candra, ST., MT
3. Dosen Penguji I, II dan III
4. Dekan Fakultas Teknik
5. Ketua Jurusan Teknik Elektro
6. Ketua Program Studi Teknik Informatika
7. Staf Dosen
8. Semua pihak yang telah membantu, penulis tidak dapat sebutkan satu persatu.

Akhir kata, penulis berharap Tuhan Yang Maha Esa berkenan membalas segala kebaikan semua pihak yang telah membantu. Semoga skripsi ini membawa manfaat bagi pengembangan ilmu.

Pekanbaru, 16 Agustus 2021

Muhamad Indra Setiabudi

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI
TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Riau, saya yang bertanda tangan di bawah ini:

Nama	: Muhamad Indra Setiabudi
NIM	: 1407114645
Program Studi	: Teknik Informatika
Departemen	: Teknik Elektro
Fakultas	: Teknik
Jenis Karya	: Skripsi

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Riau **Hak Bebas Royalti Non-Ekslusif (*Non-exclusive Royalty-Free Right*)** atas karya ilmiah saya yang berjudul:
"Implementasi Metode Frame Difference Untuk Mendeteksi Kecepatan Kendaraan Bermotor Di Universitas Riau" beserta perangkat yang ada (jika diperlukan).

Dengan Hak Bebas Royalti Non-ekslusif ini Universitas Riau berhak menyimpan, mengalih media / format, mengelola dalam bentuk pangkalan data (*database*), merawat dan mempublikasikan skripsi saya selama tetap mencantumkan nama saya sebagai penulis / pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Pekanbaru
Pada tanggal : 16 Agustus 2021
Yang menyatakan,

(Muhamad Indra Setiabudi)

Implementasi Metode *Frame Difference* Untuk Mendeteksi Kecepatan Kendaraan Bermotor Di Universitas Riau

Muhamad Indra Setiabudi
Komputasi Cerdas dan Visualisasi
Program Studi Teknik Informatika S1, Fakultas Teknik Universitas Riau

ABSTRAK

Sepeda motor merupakan kendaraan yang sudah umum kita jumpai di masa yang modern ini. Mengendarai sepeda motor dengan kecepatan yang tinggi merupakan tindakan pelanggaran sebagaimana ditunjukkan dalam Peraturan Pemerintah melalui Mentri Perhubungan (Menhub) Nomor 111 tahun 2015 tentang penetapan batas kecepatan. Berkendara dengan kecepatan tinggi juga memberikan resiko kecelakaan yang cukup tinggi. Di area kampus Universitas Riau masih sering terjadi kasus kecelakaan bermotor yang disebabkan oleh tingginya kecepatan dalam berkendara. Hal tersebut dikarenakan minimnya pengawasan di area jalanan kampus. Oleh karena itu sistem pendeteksi kecepatan motor berbasis video ini dibuat dengan harapan dapat membantu dalam mengawasi pengendara motor di kawasan Universitas Riau. Sistem yang dibuat merupakan prototype dan bukan sistem siap pakai. Sistem dibangun di dalam perangkat Raspberry pi 3 model B dengan bahasa pemrograman Python 3. Metode yang digunakan untuk mengenali objek kendaraan bermotor serta perhitungan kecepatan adalah metode *frame difference*. Penelitian dilakukan di area jalan masuk Fakultas Teknik Universitas Riau. Pengujian dilakukan dengan memperhatikan tiga faktor yang meliputi intensitas cahaya, ketinggian posisi kamera dan sudut kemiringan kamera terhadap jalan. Hasil pengujian sistem menunjukkan bahwa dalam keadaan pagi hari dengan intensitas cahaya antara 3000 Lux - 4000 Lux sistem dapat dan mencapai akurasi sebesar 93,34%. Sedangkan pada sore hari dengan intensitas cahaya antara 1500 Lux - 1600 Lux sistem dapat memberikan akurasi sebesar 86,67%. Hal tersebut menunjukkan bahwa intensitas cahaya sangat mempengaruhi hasil pengenalan yang dilakukan oleh sistem pendeteksi kecepatan.

Kata Kunci : Pengenalan Objek, Deteksi Kecepatan, Frame Difference

The Implementation Of Frame Difference Method For Motorcycle Speed Detection In University Of Riau

Muhamad Indra Setiabudi
Intelligent Computing and Visualization
S1 Informatics Engineering Study Program, Faculty of Engineering, Riau University

ABSTRACT

Motorcycles are vehicles that we commonly encounter in this modern era. Riding a motorbike at high speed is an act of violation as indicated in the Government Regulation through the Minister of Transportation (Menhub) Number 111 of 2015 concerning the setting of speed limits. Driving at high speed also poses a high risk of accidents. In the Riau University campus area there are still frequent cases of motorized accidents caused by high driving speed. This is due to the lack of supervision in the campus street area. Therefore, this video-based motorcycle speed detection system was created in the hope of helping in supervising motorcyclists in the Riau University area. The system created is a prototype and not a ready-to-use system. The system is built on a Raspberry pi 3 model B with Python 3 programming language. The method used to recognize motorized vehicles and calculate speed is the frame difference method. The research was conducted in the entrance area of the Faculty of Engineering, University of Riau. The test is carried out by three factors which include light intensity, the height of the camera position and the angle of the camera tilt to the road. The system test results show that in the morning with light intensity between 3000 Lux - 4000 Lux the system can provide excellent performance and achieve an accuracy of 93,34%. Whereas in the afternoon with light intensity between 1500 Lux - 1600 Lux the system can provide an accuracy of 86,67%. This shows that the intensity of light greatly affects the system in detecting the speed of motorized vehicles.

Keyword : *Object Recognition, Speed Detection, Frame Difference*.

DAFTAR ISI

HALAMAN PENGESAHAN ..... **iii**
HALAMAN PERNYATAAN ORISINALITAS ..... **iv**
PRAKATA ..... **v**
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS ..... **vi**
Program Studi Teknik Informatika S1, Fakultas Teknik Universitas Riau . **vii**
ABSTRAK ..... **vii**
ABSTRACT ..... **viii**
DAFTAR ISI ..... **ix**
DAFTAR GAMBAR ..... **xi**
DAFTAR TABEL ..... **xiii**
DAFTAR PERSAMAAN ..... **xiv**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan Penelitian 3
1.4 Batasan Masalah 4
1.5 Kegunaan Penelitian 4
1.6 Sistematika penulisan 5
BAB II STUDI PUSTAKA **6**
2.1 Penelitian Terkait 6
2.2 Peraturan Mentri Perhubungan Nomor PM 111 Tahun 2015 10
2.3 Video Digital 10
2.4 Pengolahan Citra Digital 12
2.5 Metode *Frame Difference* 14
2.6 *Raspberry Pi 3* Model B 16
2.7 *Python* 18

BAB III METODOLOGI 20
3.1 Metodologi Penelitian 20
3.2 Observasi 20
3.3 Studi Literatur 22
3.4 Perancangan Sistem 23
3.4.1 *Hardware* 23
3.4.2 *Software* 24
3.4.2.1 *Input* 24
3.4.2.2 *Preprocessing* 27
3.4.2.3 *Detection* 28
3.4.2.4 *Tracking* 31
3.4.2.5 *Speed Estimation* 33
3.4.2.6 *Output* 34
3.4.3 *Interface* 36
3.5 Pembuatan Sistem 37
3.6 Pengujian Sistem 37
3.6.1 Lokasi Pengujian Sistem 37
3.6.2 Tahap Pengujian Sistem 38
3.7 Analisis Hasil 38
3.8 Kesimpulan 40
BAB IV HASIL DAN PEMBAHASAN 41
4.1 Hasil 41
4.2 Analisis Hasil 49
BAB V PENUTUP 55
5.1 Kesimpulan 55
5.2 Saran 56
DAFTAR PUSTAKA 57

DAFTAR GAMBAR

Gambar 2.1 Ilustrasi Konversi Video Menjadi Kumpulan frame 11
Gambar 2.2 Citra RGB seseorang yang mengendarai sebuah sepeda motor 13
Gambar 2.3 Citra grayscale dari Gambar 2.2 13
Gambar 2.4 Citra biner dari Gambar 2.3 14
Gambar 2.5 (a) frame pertama (b) frame kedua (c) hasil perbedaan frame pertama dan kedua (d) binerisasi hasil perbedaan frame 15
Gambar 2.6 (a) frame pertama (b) frame kedua (c) hasil perbedaan frame pertama dan kedua (d) binerisasi hasil perbedaan frame 16
Gambar 2.7 Raspberry Pi 3 Model B 17
Gambar 3.1 Diagram alur penelitian 20
Gambar 3.2 Proses wawancara dengan pihak keamanan kampus 21
Gambar 3.3 Penulis dan bapak Eliyanto berfoto bersama sebagai tanda persetujuan dibangunnya prototype sistem pendeteksi kecepatan di area kampus Universitas Riau 22
Gambar 3.4 Device yang terdiri dari Raspberry pi 3 dan USB camera Logitech 23
Gambar 3.5 Alur kerja sistem 24
Gambar 3.6 Posisi objek secara realtime 25
Gambar 3.7 Posisi objek secara pada kamera 25
Gambar 3.8 Ilustrasi penempatan kamera terhadap jalan (tampak atas) 26
Gambar 3.9 Ilustrasi penempatan kamera terhadap jalan (tampak samping) 26
Gambar 3.10 Hasil ekstraksi frame dari sebuah video orang berlari 27
Gambar 3.11 Hasil konversi citra frame RGB menjadi citra frame grayscale 27
Gambar 3.12 Alur kerja metode frame difference untuk deteksi objek 28
Gambar 3.13 Ilustrasi gambar hasil frame difference 29
Gambar 3.14 Binarisasi hasil dari frame difference 30
Gambar 3.15 Pemberian bounding box pada kendaraan terdeteksi 30
Gambar 3.16 Centroid dari suatu objek yang terdeteksi pada gambar 31

Gambar 3.17 Perhitungan koordinat centroid..... 32
Gambar 3.18 Hasil centroid tracking dari suatu kendaraan pada video 33
Gambar 3.19 Bukti pelanggaran batas standar kecepatan yang dihasilkan oleh sistem 35
Gambar 3.20 Desain Antarmuka Sistem 36
Gambar 3.21 Detail lokasi pengujian sistem..... 37
Gambar 4.1 Screenshoot GUI sistem 41

DAFTAR TABEL

Tabel 2.1 Spesifikasi Raspberry Pi 3 Model B 17
Tabel 3.1 Keterangan Gambar 3.19..... 36
Tabel 4.1 Data hasil pengujian deteksi deteksi kecepatan pada speedometer 30 km/j 47
Tabel 4.2 Data hasil pengujian deteksi deteksi kecepatan pada speedometer 40 km/j 47
Tabel 4.3 Data hasil pengujian deteksi deteksi kecepatan pada speedometer 45 km/j 48
Tabel 4.4 Analisis hasil kinerja sistem pada keadaan pagi hari 49
Tabel 4.5 Analisis hasil kinerja sistem pada keadaan sore hari 50
Tabel 4.6 Analisis kinerja sistem secara keseluruhan 50
Tabel 4.7 Hasil pendeteksian status pelanggaran pada kecepatan 30 km/j 51
Tabel 4.8 *Multiclass confusion matrix* pada kecepatan 30 km/j 52
Tabel 4.9 Hasil pendeteksian status pelanggaran pada kecepatan 40 km/j 52
Tabel 4.10 Multiclass confusion matrix pada kecepatan 40 km/j 52
Tabel 4.11 Hasil pendeteksian status pelanggaran pada kecepatan 45 km/j 53
Tabel 4.12 Multiclass confusion matrix pada kecepatan 45 km/j 53
Tabel 4.13 Hasil pendeteksian status pelanggaran secara keseluruhan..... 54
Tabel 4.14 Multiclass confusion matrix hasil deteksi keseluruhan..... 54

DAFTAR PERSAMAAN

(2.1) Rumus Frame Difference 15
(3.1) Rumus Kecepatan..... 33
(3.2) Rumus Jarak Euclidean 34
(3.3) Rumus Deteksi Kecepatan..... 34
(3.4) Rumus Akurasi Confusion Matrix..... 39

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sepeda motor merupakan jenis kendaraan yang umum kita jumpai di masa modern ini. Harga yang semakin terjangkau serta kegunaannya dalam membantu motoritas manusia menyebabkan penyebaran kendaraan satu ini menjadi begitu pesat. Bukan hanya dari kalangan orang tua saja, bahkan anak-anak muda di masa kini sudah banyak yang memiliki kendaraan bermotor untuk menunjang kehidupan sehari-harinya. Tenaga yang dapat dihasilkan oleh mesin motor di masa kini juga tergolong besar sehingga mampu membuat motor dipacu dengan kecepatan yang tinggi pula.

Mengendarai sepeda motor dengan kecepatan diatas standar keselamatan merupakan sebuah pelanggaran terhadap peraturan pemerintah dan dapat membahayakan orang lain maupun diri sendiri. Salah satu penyebab utama dalam terjadinya kecelakaan dalam berlalulintas adalah berkendara dengan kecepatan tinggi yang menyebabkan kendaraan sulit untuk dikendalikan pada keadaan tertentu. Dalam upaya mengurangi tingkat kecelakaan lalulintas yang disebabkan oleh pengendara berkecepatan tinggi, pemerintah sudah mengeluarkan peraturan melalui Mentri Perhubungan (Menhub) Nomor 111 tahun 2015 tentang penetapan batas kecepatan (Perhubungan, 2015). Namun minimnya peralatan yang mampu membantu dalam pendeteksian kecepatan pengendara dalam berkendara menjadi sebuah kekurangan dalam memaksimalkan peraturan tersebut terutama di area kampus perkuliahan.

Di area kampus perkuliahan khususnya Universitas Riau sering ditemukan pengendara yang memacu kendaraannya dengan kecepatan yang relatif tinggi dan berpotensi membahayakan pengendara lain maupun si pengendara itu sendiri. Padahal di area kampus banyak sekali pejalan kaki maupun pesepeda yang berpotensi menjadi korban kecelakaan lalu lintas akibat berkendara dengan

kecepatan tinggi. Hal tersebut dapat dibuktikan dengan adanya beberapa kejadian kecelakaan baik itu tabrakan antar kendaraan ataupun kecelakaan tunggal di area kampus Universitas Riau. Bukan hanya potensi kecelakaan saja, bahkan suara keras yang dihasilkan oleh kendaraan yang melaju kencang juga dapat mengganggu ketenangan area perkuliahan.

Usaha penertiban pengendara bermotor di kawasan Universitas Riau sudah dilakukan oleh pihak keamanan kampus dengan cara memasang rambu-rambu batas maksimal kecepatan yaitu 40 kilometer per jam (km/j) di beberapa titik jalan Universitas Riau. Namun dalam pengaplikasiannya masih sulit untuk dilakukan karena masih banyaknya titik tanpa pengawasan. Hal tersebut dikarenakan minimnya fasilitas yang dapat mendukung serta jumlah sumber daya manusia di area tersebut.

Melihat beberapa permasalahan di atas maka penertiban pengendara sepeda motor di area kampus Universitas Riau dengan sebuah sistem yang mampu mendeteksi kecepatan berkendara perlu direalisasikan. Salah satu sistem pendeteksi kecepatan yang efektif untuk menangani beberapa masalah di atas adalah sebuah sistem pendeteksi kecepatan berbasis video. Kemampuan yang diunggulkan dari sistem pendeteksi kecepatan berbasis video adalah tidak memerlukan sumber daya manusia atau operator secara langsung di area yang sudah terawasi oleh kamera. Hal tersebut dikarenakan sebuah sistem pendeteksi kecepatan berbasis video hanya memerlukan kamera statis yang diletakkan di area yang dituju sedangkan proses pendeteksian kecepatan pada video dilakukan secara otomatis oleh sistem dan hasil dari pendeteksian dapat langsung dikirim ke komputer *server*.

Sistem pendeteksi kecepatan berbasis video didasari oleh pendeteksian objek bergerak pada video. *Frame difference* merupakan metode yang umum digunakan dalam pendeteksian objek bergerak dalam sebuah video (Singla, 2014). Pendeteksian terhadap objek bergerak oleh metode ini terhitung cukup baik. Metode ini mendeteksi objek bergerak dengan cara membandingkan nilai setiap piksel antar *frame* secara berurut guna mendapatkan selisih nilai piksel yang diasumsikan sebagai objek bergerak. Hasil dari perhitungan selisih piksel antar *frame* tersebut akan diproses dengan metode pengolahan citra digital seperti

mengubah nilai setiap piksel dalam gambar menjadi 0 atau 1 (*binarization*) agar luas dari setiap objek yang terdeteksi dapat dihitung. Hasil perhitungan luas objek yang terdeteksi bertujuan untuk menghilangkan objek – objek yang tidak diinginkan (*noise*) yang pada umumnya memiliki luas yang jauh lebih kecil ataupun jauh lebih besar dari objek yang dituju. Hal tersebut dilakukan untuk meningkatkan keakurasian dari hasil pendeteksian objek bergerak pada video.

Dari acuan yang ada di atas maka penulis melakukan penelitian dengan judul *Implementasi Metode Frame Difference untuk Mendeteksi Kecepatan Kendaraan Bermotor di Universitas Riau* ini supaya dapat menjadi pendukung bagi pihak keamanan dalam menertibkan pengendara motor di area kampus Universitas Riau. Penelitian ini dilakukan dengan menggunakan metode *frame difference* untuk mendeteksi objek dan menghitung kecepatan kendaraan bermotor. Jika hasil dari perhitungan kecepatan melebihi standar kecepatan yang telah ditetapkan, maka sistem secara otomatis akan menyimpan bukti pelanggaran berupa foto terkait kendaraan tersebut.

1.2 Rumusan Masalah

Dalam penulisan skripsi ini ada beberapa hal yang akan dibahas, diantaranya yaitu:

1. Bagaimana cara mendeteksi sepeda motor bergerak pada sebuah video yang ditangkap oleh kamera digital?
2. Bagaimana cara menghitung kecepatan pergerakan sepeda motor pada sebuah video yang ditangkap oleh kamera digital?

1.3 Tujuan Penelitian

Tujuan dari dilakukannya penelitian ini adalah untuk merancang sebuah sistem yang dapat mendeteksi kecepatan pergerakan kendaraan bermotor dalam sebuah video menggunakan metode *frame difference* serta menyimpan foto bukti

pelanggaran bila kendaraan yang dideteksi melebihi batas standar kecepatan yang telah ditentukan di area kampus Universitas Riau.

1.4 Batasan Masalah

Dalam penulisan skripsi ini, berbagai batasan ditetapkan agar memudahkan pembahasan dan memfokuskan pada hal yang ingin diteliti saja. Adapun batasan yang ditetapkan adalah:

1. Objek diambil menggunakan kamera digital Logitech C310 dengan resolusi 5 megapiksel (MP).
2. Sistem dibangun menggunakan Raspberry Pi 3 Model B dengan bahasa pemrograman *Python* 3.
3. Sistem yang dibangun merupakan *prototype*.
4. Objek yang dideteksi merupakan *single object*.
5. Pemberian detail informasi pada foto bukti pelanggaran menggunakan *library Pillow*.
6. Waktu pengambilan sampel hanya pada siang hari.
7. Kendaraan yang dideteksi kecepatannya hanya kendaraan berjenis sepeda motor.
8. Wilayah penelitian adalah area jalan masuk Fakultas Teknik Universitas Riau.

1.5 Kegunaan Penelitian

Adapun kegunaan penelitian ini dilakukan adalah :

1. Menghasilkan suatu protoype sistem pendeteksi kecepatan sebuah sepeda motor berbasis video.
2. Membantu pihak keamanan kampus dalam mendeteksi tingkat pelanggaran standar kecepatan di area kampus Universitas Riau.

1.6 Sistematika penulisan

Struktur penulisan skripsi ini tersusun dalam lima bab dengan sistematika penulisan sebagai berikut:

BAB I : PENDAHULUAN

Bab ini membahas tentang latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, kegunaan penelitian, dan sistematika penulisan.

BAB II : STUDI PUSTAKA

Bab ini berisi landasan-landasan teori yang berkaitan dengan penelitian yang penulis lakukan.

BAB III : METODOLOGI

Bab ini berisi tentang tahapan-tahapan yang dilakukan pada penelitian mulai dari metodologi penelitian, tahap perancangan sistem, tahap pembuatan sistem, tahap pengujian, tahap analisis hasil pengujian, sampai pada kesimpulan.

BAB IV : IMPLEMENTASI DAN ANALISA HASIL

Bab ini berisi tentang implementasi dan analisis dari sistem yang dibuat mulai dari implementasi sesuai perancangan sistem, pengujian sistem, analisi hasil pengujian sistem dan penarikan kesimpulan.

BAB V : PENUTUP

Bab ini berisi tentang jawaban singkat dari rumusan masalah atau kesimpulan dan saran dalam pengembangan serta penyempurnaan penelitian yang dibuat untuk kedepannya.

BAB II
STUDI PUSTAKA

2.1 Penelitian Terkait

Penelitian mengenai deteksi objek bergerak dengan judul *Motion Detection Based on Frame Difference Method* (Singla, 2014) dilakukan untuk membuktikan apakah metode *frame difference* dapat mendeteksi objek bergerak dalam sebuah video dengan cukup baik atau tidak. Dalam penelitian tersebut *sample* yang digunakan adalah sebuah rekaman video dengan posisi kamera diam atau statis dan latar belakangnya adalah sebuah ruangan yang tetap atau statis pula. Berikutnya peneliti berjalan melewati kamera di dalam ruangan tersebut sebagai objek bergerak di dalam video. Setelah itu video diproses menggunakan metode *frame difference* untuk menghitung perbedaan piksel antar *frame*. Berikutnya hasil dari *frame difference* akan di-*convert* ke dalam *gray* dan *binary image* dan terakhir dilakukan proses mhorpologi untuk menghilangkan *noise*. Hasil dari penelitian ini menunjukan bahwa metode *frame difference* dapat mengenali objek bergerak pada sebuah video dengan cukup baik.

Berikutnya penelitian dengan judul *Sistem Pengukur Kecepatan Kendaraan Berbasis Pengolahan Video* (Sadewo et al., 2015) membahas tentang pengukuran kecepatan kendaraan pada video dengan metode *Gaussian Mixture Model* (GMM). Penelitian dilakukan dengan cara menempatkan kamera statis di sisi kanan area pengujian untuk menangkap kendaraan yang bergerak di area uji. Berikutnya video yang telah didapat diproses menggunakan metode GMM untuk mendeteksi adanya objek pada video dan menghitung kecepatannya. Hasil dari penelitian ini menunjukkan pengukuran kecepatan terbaik dengan persentase rentang nilai *error* antara 1,92% - 15,75% pada keadaan pagi, siang dan sore hari dengan jangkauan intensitas cahaya antara 600 – 1900 lux, sedangkan pada malam hari yang memiliki jangkauan intensitas cahaya antara 0 – 5 lux objek tidak dapat terdeteksi.

Penelitian mengenai pengukuran kecepatan kendaraan juga dilakukan oleh Romli (2015) dengan judul *Pengukuran Kecepatan Kendaraan Menggunakan Optical Flow*. Dalam penelitian ini metode yang diangkat adalah *Optical Flow*. Penelitian dilakukan dengan menempatkan kamera statis di salah satu sisi sebuah jembatan penyebrangan yang mengarah ke jalan raya. Uji coba dilakukan menggunakan 11 *sample* video dengan 2 relosuli berbeda yaitu 9 video dengan resolusi 720x1280 *pixel* dan 2 video dengan resolusi 480x640 *pixel* dan diambil pada keadaan pagi dan sore hari. Hasil dari pengujian menunjukan perhitungan dengan akurasi terendah sebesar 74,84% dan perhitungan dengan akurasi terbaik sebesar 89,37%.

Penelitian dengan judul *Deteksi Objek Bergerak Pada Video Bawah Air Menggunakan Metode Frame Difference* (Muhamad et al., 2019) menguji kecakapan metode *frame difference* dalam mendeteksi objek bawah air. Penelitian dilakukan dengan menggunakan dua buah kamera yang dikalibrasikan untuk menghasilkan satu buah video dengan kualitas yang baik. Kamera yang telah dipersiapkan dengan sedemikian rupa dibawa ke dalam air sehingga dapat menangkap objek bergerak di bawah air. Video hasil rekeman diproses menggunakan metode *frame difference* dengan tingkat ketepatan mencapai 96% dalam mendeteksi objek bergerak yang pada penelitian ini adalah salah satwa laut yaitu ikan. Hal tersebut menunjukkan kecakapan metode *frame difference* dalam mendeteksi objek bawah air tergolong sangat baik.

Penelitian dengan judul *Perancangan Aplikasi Deteksi Kemacetan Berdasarkan Pengolahan Video Digital Menggunakan Metode Frame Difference Berbasis Android* (Marieski et al., 2017) menunjukkan bahwa metode *frame difference* bekerja dengan baik untuk identifikasi objek bergerak. Hal tersebut ditunjukkan oleh hasil dari penelitian ini yang memberikan tingkat keakurasian mencapai 90% untuk kondisi pagi hari, 85% pada siang hari dan 75% untuk kondisi sore hari. Penelitian ini dilakukan dengan cara membangun sebuah aplikasi pemrosesan video pada android dengan menggunakan metode *frame difference* sebagai teknik pengenalan objek bergerak untuk identifikasi kemacetan di salah satu ruas jalan.

Penelitian dengan judul *Analisa Gerakan Manusia Pada Video Digital* (Rohman n.d. 2011) meneliti tentang teknik pendeteksian objek bergerak pada video digital dan menentukan jarak perpindahan objek dengan menggunakan metode *background substraction* dan *frame difference*. Hasil dari penelitian ini menunjukkan bahwa posisi sudut kamera terhadap objek bergerak dalam pengambilan video sangat mempengaruhi tingkat keberhasilan validasi untuk pendeteksian dan perhitungan jarak perpindahan objek pada video. Selain posisi sudut kamera terhadap objek, gerakan yang dilakukan oleh objek serta jumlah objek yang terdeteksi di dalam video juga mempengaruhi tingkat keberhasilan dan keakuratan validasi.

Penelitian dengan judul *Moving Object Detection Using Frame Difference, Background Substraction And SOBS For Video Surveillance Aplication* (Gupta et al., 2014) meneliti perbandingan tingkat keefektifan metode *frame difference, background substraction* dan *neural mapping methods* (SOBS) dalam mendeteksi objek bergerak pada sebuah video. Penelitian dilakukan dengan cara menerapkan ketiga teknik deteksi objek bergerak tersebut pada satu buah video secara bergantian. Hasil dari perbandingan ketiga metode menunjukkan bahwa metode *background substraction* dengan *dynamic threshold* adalah metode yang paling cocok untuk mendeteksi objek bergerak pada video.

Penelitian dengan judul *Three-Frame Difference Algorithm Research Based on Mathematical Morphology* (Zhang et al., 2012) meneliti tentang pengembangan metode *three-frame difference* dengan menambahkan teknik *mathematical morphology*. Penelitian dilakukan dengan cara mendeteksi objek bergerak pada video terlebih dahulu menggunakan metode *three-frame difference*. Setelah citra biner yang merupakan hasil selisih antar *frame* didapatkan, maka berikutnya teknik *mathematical morphology* yaitu dilasi dan erosi diaplikasikan kepada citra biner guna menambah kualitas serta mengurangi noise citra tersebut. Hasil penelitian menunjukkan bahwa metode yang diangkat pada penelitian ini memberikan kinerja yang lebih baik berupa lebih singkatnya waktu pemrosesan, meningkatnya performa pengenalan secara *real-time* dan akurasi yang lebih tinggi serta dapat diaplikasikan di luar ruangan.

Penelitian dengan judul *Analisis dan Deteksi Gerakan Kepanikan Menggunakan Metode Frame Difference pada Sistem Monitoring Rumah* (Nuralim et al., 2019) dilakukan dengan tujuan untuk mendeteksi tingkat kepanikan seseorang di dalam rumah berdasarkan tingkat kecepatan gerakan orang tersebut. Metode yang diusung pada penelitian ini adalah metode *frame difference*. Penelitian dilakukan dengan menempatkan sebuah kamera statis di dalam sebuah ruangan guna menangkap pergerakan orang yang berada di dalam ruangan tersebut. Berikutnya hasil rekaman akan diproses dengan metode *frame difference* untuk mendeteksi pergerakan dan kecepatan pergerakan orang yang ada di dalam video. Hasil pendeteksian akan dibandingkan dengan kecepatan perbandingan guna mengetahui apakah orang yang ada di dalam video sedang panik atau tidak. Hasil dari penelitian menunjukkan akurasi tertinggi mencapai 85.71%.

Penelitian mengenai pendeteksian objek bergerak dengan judul *BFSD: Background Substraction Frame Difference Algorithm for Moving Object Detection and Extraction* (Alex & Wahi, 2014) dilakukan dengan menggabungkan dua metode guna melahirkan sebuah algoritma alternatif untuk pendeteksian objek bergerak yang diberi nama *Background Substraction Frame Difference* (BFSD). Pendeteksian objek bergerak dilakukan dengan cara menerapkan metode *frame difference* terlebih dahulu untuk mendapatkan *frame* referensi atau *frame background*, setelah itu metode *background substraction* diimplementasikan guna menemukan objek bergerak pada *frame* dan mengekstraknya. Hasil dari penelitian menunjukkan pendeteksian objek bergerak dengan tingkat keefektifan dan akurasi yang lebih tinggi dibandingkan metode *frame difference* maupun metode *background substraction*.

2.2 Peraturan Mentri Perhubungan Nomor PM 111 Tahun 2015

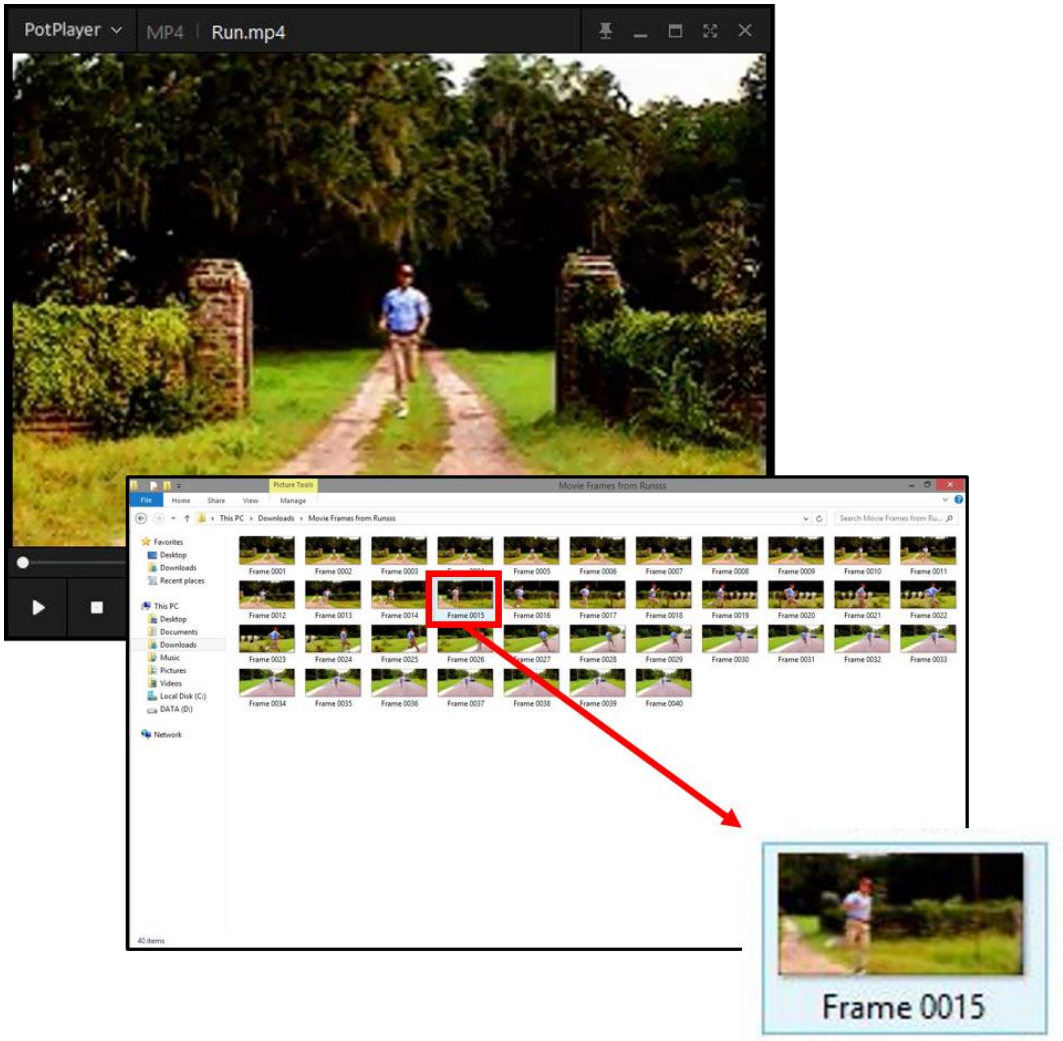
Aturan penentuan batas kecepatan dalam berkendara secara resmi telah dikeluarkan oleh Pemerintah melalui Mentri Perhubungan yaitu Peraturan Mentri Perhubungan nomor PM 111 tahun 2015. Pada pasal 3 ayat 4 disebutkan detail besaran batas kecepatan berkendara dalam satuan kilometer per jam (km/j) di 4 keadaan jalan berbeda sebagai berikut.

a) Paling rendah 60 km/j dan paling tinggi 100 km/j untuk keadaan jalan bebas hambatan.
b) Paling tinggi 80 km/j untuk keadaan jalan antar kota.
c) Paling tinggi 50 km/j untuk keadaan jalan kawasan perkotaan.
d) Paling tinggi 30 km/j untuk keadaan jalan area permukiman.

2.3 Video Digital

Video dapat disebut juga sebagai citra bergerak (*moving object*) merupakan rangkaian citra diam (gambar) yang ditampilkan secara beruntun dalam kurun waktu tertentu sehingga menghasilkan kesan gambar yang bergerak pada mata kita. Setiap gambar yang disusun demi menciptakan sebuah video disebut dengan *frame*. Film layar lebar atau tayangan pada televisi yang sering kita saksikan pada dasarnya merupakan kumpulan dari ratusan sampai ribuan *frame* yang ditampilkan secara beruntun.

Kualitas dari sebuah video ditentukan oleh banyaknya *frame* yang dijalankan per detiknya atau disebut juga *frame per second* (FPS). Semakin banyak *frame* yang dijalankan dalam 1 detiknya maka semakin baik pula kualitas dari video tersebut. Semakin banyak jumlah FPS memang akan membuat kualitas video semakin baik, namun hal tersebut juga diikuti oleh meningkatnya ukuran file dari video tersebut. Hal tersebut dikarenakan jumlah *frame* yang digunakan untuk membangun sebuah video juga bertambah banyak. Untuk lebih jelasnya dapat dilihat ilustrasi hasil konversi video ke dalam kumpulan *frame* pada Gambar 2.1.



Gambar 2.1 Ilustrasi Konversi Video Menjadi Kumpulan frame

Video yang diilustrasikan di atas merupakan sebuah video orang sedang berlari dengan durasi 4 detik dan total *frame* sebanyak 40 *frame*. Hal tersebut menyebabkan konversi dari video di atas menghasilkan 40 *frame* pada ruang penyimpanan. *Frame* hasil konversi merupakan sebuah citra diam seperti yang ditunjukkan oleh tanda panah merah pada Gambar 2.1 yang merupakan salah satu *frame* yang diperbesar. Setiap *frame* hasil konversi video dapat diolah sedemikian rupa dengan berbagai metode pengolahan citra digital untuk kebutuhan penelitian ataupun kebutuhan di bidang lainnya.

2.4 Pengolahan Citra Digital

Citra atau yang sering kita kenal dengan kata “gambar” dapat diartikan sebagai suatu fungsi intensitas cahaya dua dimensi, yang dinyatakan oleh *f(x,y)*, di mana nilai atau amplitudo dari *f* pada koordinat spasial *(x,y)* menyatakan intensitas (kecerahan) citra pada titik tersebut. Agar citra yang mengalami gangguan mudah diinterpretasi (baik oleh manusia maupun mesin), maka citra tersebut perlu dimanipulasi menjadi citra lain yang kualitasnya lebih baik. Bidang studi yang menyangkut hal ini adalah pengolahan citra (*image processing*) (Gonzales et al., 2004).

Dari pernyataan di atas, pengolahan citra dapat didefinisikan sebagai ilmu untuk memanipulasi gambar, yang melingkupi teknik-teknik untuk memperbaiki atau mengurangi kualitas gambar, menampilkan bagian tertentu dari gambar, membuat sebuah gambar yang baru dari beberapa bagian gambar yang sudah ada, dan beberapa teknik manipulasi gambar lainnya, pemrosesan citra, khususnya dengan menggunakan komputer, menjadi citra yang kualitasnya lebih baik.

Pada aplikasi pengolahan citra digital pada umumnya, citra digital dapat dibagi menjadi 3, *color image, black and white image* dan *binary image*.

1. *Color Image* atau RGB (*Red, Green, Blue*).

Pada *color image* ini masing-masing piksel memiliki warna tertentu, warna tersebut adalah merah (*Red*), hijau (*Green*) dan biru (*Blue*). Jika masing-masing warna memiliki range 0 - 255, maka totalnya adalah 255 3 = 16.581.375 (16 K) variasi warna berbeda pada gambar, dimana variasi warna ini cukup untuk gambar apapun. Karena jumlah bit yang diperlukan untuk setiap pixel, gambar tersebut juga disebut gambar-bit warna. Contoh citra RGB dari seseorang yang mengendarai sepeda motor dapat dilihat pada Gambar 2.2.



Gambar 2.2 Citra RGB seseorang yang mengendarai sebuah sepeda motor

2. *Black and White image*.

Citra *black and white* (*grayscale*) merupakan sebuah citra digital yang setiap pikselnya mempunyai warna gradasi mulai dari putih sampai hitam. Rentang tersebut berarti bahwa setiap piksel dapat diwakili oleh 8 bit, atau 1 byte. Rentang warna pada *black and white* sangat cocok digunakan untuk pengolahan file gambar. Salah satu bentuk fungsinya digunakan dalam pendeteksian objek bergerak pada sebuah video. Contoh citra hitam putih dari Gambar 2.2 dapat dilihat pada Gambar 2.3.



Gambar 2.3 Citra *grayscale* dari Gambar 2.2

3. *Binary image*

Setiap piksel hanya terdiri dari warna hitam atau putih, karena hanya ada dua warna untuk setiap piksel, maka hanya perlu 1 bit per piksel (0 dan 1) atau apabila dalam 8 bit (0 dan 255), sehingga sangat efisien dalam hal penyimpanan. Gambar yang direpresentasikan dengan biner sangat cocok untuk teks (dicetak atau tulisan tangan), sidik jari (*finger print*), atau gambar arsitektur. Contoh citra biner dari Gambar 2.3 dapat dilihat pada Gambar 2.4.



Gambar 2.4 Citra biner dari Gambar 2.3

2.5 Metode *Frame Difference*

Frame difference merupakan sebuah metode yang digunakan untuk mendeteksi objek bergerak dalam sebuah video. Metode ini mengenali adanya objek bergerak dengan cara menghitung selisih setiap *pixel* antara *frame* sekarang (f_t) dengan 1 *frame* sebelumnya (f_{t-1}) dan membandingkannya dengan nilai *Threshold* yang ditentukan sehingga menghasilkan sebuah *image* differensial I_d.

Teknik perhitungan metode *frame difference* dapat dilihat pada rumus berikut:

$$I_{d(t.t-1)} = | I_t - I_{t-1}| \quad (2.1)$$

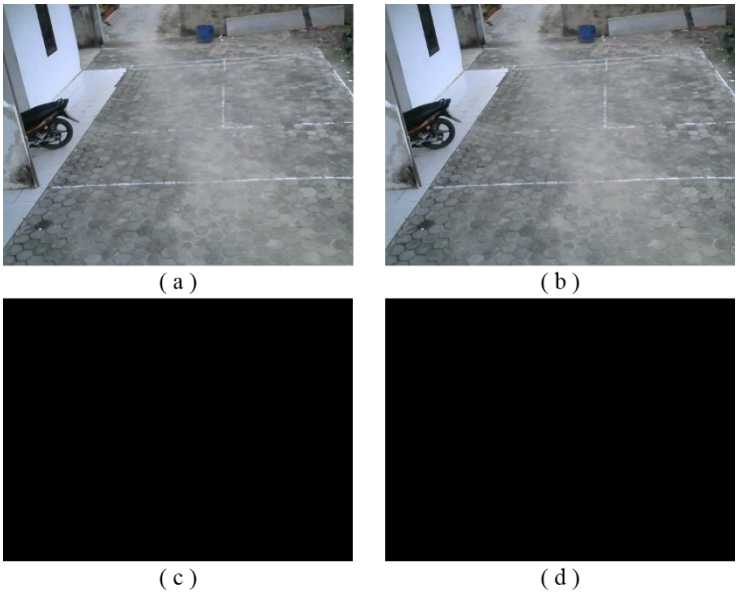
Keterangan :

I_d : *Image* differensial

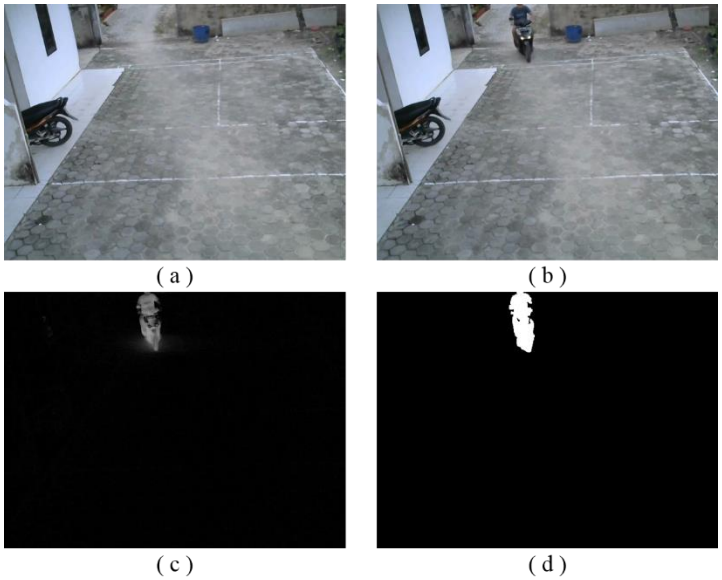
I_t : *Image* pada *frame* sekarang

I_{t-1} : *Image* pada *frame* sebelumnya

Citra hasil dari differensial yang didapatkan masih menyimpan banyak *noise* sehingga perlu dikonversikan ke dalam citra biner agar dapat dilakukan operasi penghapusan *noise* dan dapat diberi identitas sebagai objek bergerak dalam sebuah video. Untuk lebih jelasnya ilustrasi mengenai penerapan metode *frame difference* dapat dilihat pada Gambar 2.5 untuk *frame* tanpa adanya objek bergerak dan Gambar 2.6 untuk *frame* dengan adanya objek bergerak.



Gambar 2.5 (a) frame pertama (b) frame kedua (c) hasil perbedaan frame pertama dan kedua (d) binerisasi hasil perbedaan frame



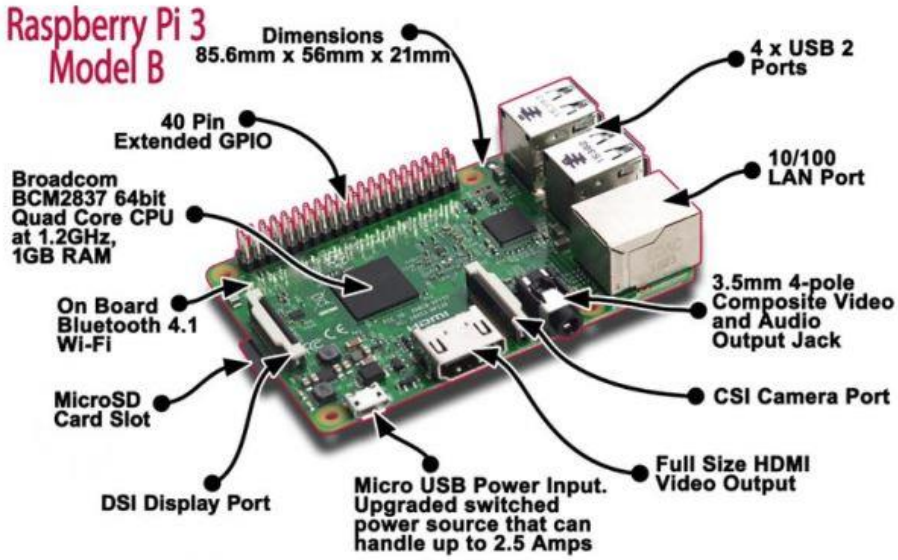
Gambar 2.6 (a) frame pertama (b) frame kedua (c) hasil perbedaan frame pertama dan kedua (d) binerisasi hasil perbedaan frame

2.6 *Raspberry Pi 3* Model B

Raspberry Pi atau dikenal juga dengan sebutan raspi merupakan sebuah modul mikro komputer yang memiliki *digital port input output* (GDIO) seperti mikrokontroller pada umumnya. Meskipun terlihat mirip, raspi memiliki kelebihan tersendiri dibandingkan dengan mikrokontroller lain yaitu adanya *built-in display port* untuk koneksi ke perangkat TV atau monitor PC. Selain itu raspi juga memiliki *built-in* USB *port* untuk koneksi ke perangkat *keyboard* dan *mouse* yang tidak dimiliki oleh mikrokontroller lain.

Raspberry Pi 3 merupakan generasi ke-3 dalam seri *Raspberry Pi* yang diciptakan untuk menggantikan pendahulunya yaitu *Raspberry Pi 2* dan *Raspberry Pi 1*. Keunggulan yang signifikan antara raspi 3 dengan model-model raspi sebelumnya adalah adanya *built-in wireless connection* dan *processor* yang lebih kuat. Raspi 3 sendiri memiliki 2 model yaitu model A dan model B. Perbandingan antara model A dan model B terletak pada kapasitas *Random Access Memory* (RAM) model B yang lebih besar dari pada kapasitas RAM model A serta terdapatnya *Ethernet port* (untuk koneksi LAN) pada model B dan tidak ada pada model A (Cilsy, 2018). Bentuk dari *Raspberry Pi 3* model B dapat dilihat pada

Gambar 2.7 sedangkan spesifikasi dari perangkat *Raspberry Pi 3* model B dapat dilihat pada Tabel 2.1.



Gambar 2.7 Raspberry Pi 3 Model B

Tabel 2.1 Spesifikasi Raspberry Pi 3 Model B

Spesifikasi	Keterangan
Soc	BCM2837
Processor	1.2GHz 64-bit quad-core ARMv8 CPU
RAM	1 GB SDRAM 400MHz
GPU	VideoCore IV 3D graphics core
LAN	802.1 In Wireless LAN
Bluetooth	Bluetooth 4.1 (built-in), Bluetooh Low Energy (BLE)
GPIO	40 Pin
Port USB	4 USB port
Card Storage	Micro SD card slot
Connection	Ethernet port
External Audio and Video	Full HDMI port, Camera Interface (CSI), Display Interface (DSI), Combined 3.5mm audio jack and composite video
Operating System	Raspbian, Debian GNU/Linux, Fedora, Arch Linux ARM, RISC OS

2.7 *Python*

Python merupakan sebuah bahasa pemrograman tingkat tinggi atau *high-level programming language*. Dalam dunia pemrograman, semakin tinggi tingkatan sebuah bahasa pemrograman maka kode-kode yang digunakan dalam bahasa pemrograman tersebut semakin dekat dengan bahasa manusia lebih tepatnya bahasa inggris. Semakin dekat sebuah kode bahasa pemrograman dengan bahasa manusia maka semakin mudah untuk dipahami dan diaplikasikan dalam penulisan kode program.

Pada awalnya *Python* dibuat dan dikembangkan oleh seorang *programmer* asal Belanda bernama Guido Van Rossum pada tahun 1990. Pada tahun 1995 *Python* kembali dikembangkan oleh Guido Van Rossum agar lebih kompatibel terhadap *device* yang ada. Kemudian pada tahun 2000 bahasa pemrograman *Python* terus dikembangkan hingga kini mencapai versi ke-3 (Ilham, 2019).

Python adalah bahasa pemrograman yang bersifat *interpreted* dimana baris kode akan diproses pada saat *runtime* oleh interpreter secara langsung sehingga tidak perlu melakukan kompilasi terlebih dahulu, *interactive* yang artinya kita dapat berinteraksi secara langsung dengan *Python* melalui *Prompt Python* pada saat *coding*, dan *object-oriented* karena *Python* mendukung sistem pemrograman berbasis objek atau *object-oriented programming*.

Sebagai sebuah bahasa pemrograman tingkat tinggi, *Python* memiliki beberapa kelebihan sekaligus kekurangan. Berikut beberapa kelebihan yang terdapat pada bahasa pemrograman *Python* :

1) Mudah dipelajari karena bahasanya yang sudah mendekati bahasa manusia (bahasa inggris).
2) Mendukung *Internet of Things* (IoT) dengan sangat baik. IoT merupakan kemampuan menghubungkan benda-benda di sekitar kita ke dalam sebuah jaringan untuk dapat berkomunikasi antara satu dengan yang lain.
3) Mampu mengembangkan sebuah program dengan lebih efektif karna *coding* yang lebih sedikit dan mudah dimengerti.
4) *Library* yang luas dan banyak.

5) Mempunyai sistem pengelola memory yang otomatis (*grabage collection*) seperti *Java*.
6) Memiliki modul yang selalu diperbarui.
7) Mendukung multi *platform* dan multi sistem. Saat ini *Python* dapat digunakan untuk membuat aplikasi di beberapa sistem operasi berikut:
    a. Linux.
    b. Microsoft Windows.
    c. Mac OS.
    d. Android.
    e. Java Virtual Machine.
    f. Symbian OS.
    g. Amiga.
    h. Palm.
    i. OS/2.

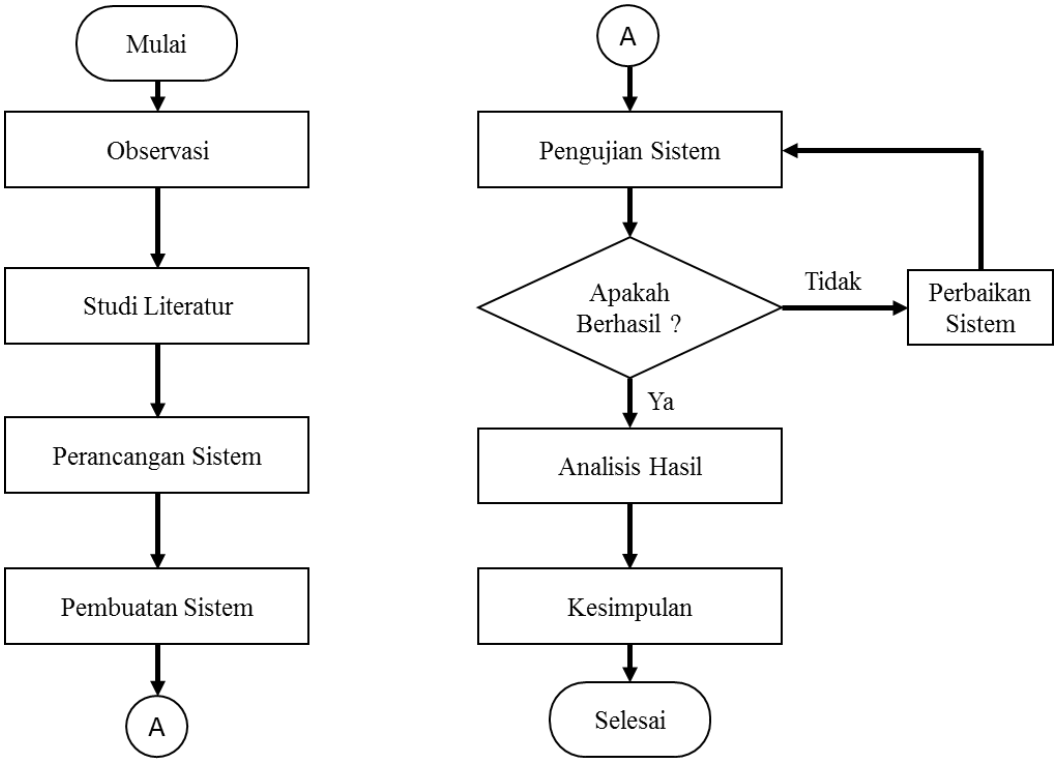
Disamping berbagai kelebihan yang ditawarkan oleh *Python*, beberapa kekurangna juga dimiliki oleh bahasa pemrograman ini sebagai berikut:

a. Eksekusi *coding* lambat.
b. Lemah dalam hal komputasi *motore*.
c. Lemah dalam pemrograman grafis 3D.
d. Kurang efisien untuk program yang memerlukan kinerja *multi-core*.
e. Terbatas dalam akses basis data (IWD, 2019).

BAB III
METODOLOGI

3.1 Metodologi Penelitian

Bagian ini berisi uraian tentang alur penelitian secara umum yang dapat dilihat pada Gambar 3.1.



Gambar 3.1 Diagram alur penelitian

3.2 Observasi

Langkah pertama yang dilakukan oleh penulis dalam penelitian ini adalah melakukan observasi guna mengetahui permasalahan mengenai penyebab utama kecelakaan sepeda motor di area kampus Universitas Riau serta memberikan solusi berbasis teknologi. Observasi penulis lakukan dengan cara melakukan wawancara

dengan kepala bagian keamanan kampus Universitas Riau yaitu bapak Eliyanto. Dalam proses wawancara, penulis meminta izin kepada bapak Eliyanto untuk melihat rekap data kecelakaan kendaraan bermotor di Universitas Riau serta membahas detail data rekap yang diberikan. Proses wawancara dapat dilihat pada Gambar 3.2.



Gambar 3.2 Proses wawancara dengan pihak keamanan kampus

Setelah melihat rekap data kejadian kecelakan sepeda motor di area kampus Universitas Riau, penulis menemukan bahwa jumlah kejadian kecelakaan yang terdata cukup banyak. Bapak Eliyanto mengatakan bahwa penyebab utama banyaknya terjadi kecelakaan adalah kecepatan berkendara yang melebihi kecepatan standar yang sudah ditentukan yaitu 40 km/j. Sedangkan banyaknya kasus pelanggaran batas kecepatan oleh pengendara terjadi karena masih kurangnya pengawasan di area kampus. Beliau menyatakan bahwa usaha penertiban di beberapa titik area jalanan kampus sudah dilakukan dengan cara memasang tanda batas kecepatan maksimal dan polisi tidur. Namun kurangnya fasilitas pendukung dan jumlah SDM yang tersedia menjadi kekurangan yang sangat berpengaruh dalam menertibkan pengendara di area kampus.

Banyaknya kejadian kecelakaan yang terjadi dan minimnya fasilitas pendukung dalam menertibkan pengendara menjadikan inisialisasi sistem pendeteksi kecepatan kendaraan bermotor berbasis teknologi terutama kamera yang dapat menyimpan bukti pelanggaran perlu dilakukan. Hal tersebut dikarenakan dengan hadirnya sistem ini diharapkan dapat menambah kekuatan pihak keamanan kampus dalam melakukan pengawasan di area kampus Universitas Riau. Bapak Eliyanto selaku kepala bidang keamanan kampus Universitas Riau juga menyetujui pembangunan *prototype* sistem pendeteksi kecepatan ini. Hal tersebut dapat dilihat dari Gambar 3.3.



Gambar 3.3 Penulis dan bapak Eliyanto berfoto bersama sebagai tanda persetujuan dibangunnya prototype sistem pendeteksi kecepatan di area kampus Universitas Riau

3.3 Studi Literatur

Studi literatur dilakukan untuk mengetahui konsep-konsep dasar yang menunjang penelitian ini. Literatur yang dikaji pada penelitian ini berhubungan dengan Penelitian terdahulu mengenai pendeteksian kecepatan objek pada sebuah video, peraturan Mentri Perhubungan nomor PM 111 tahun 2015 tentang penentuan

batas kecepatan, Video Digital, Pengolahan Citra Digital, Metode *Frame Difference*, *Raspberry Pi* 3 Model B, dan *Python*.

3.4 Perancangan Sistem

Tahap perancangan sistem dilakukan agar sistem yang akan dibuat memiliki alur dan tujuan yang jelas serta meminimalisir adanya kekurangan serta kelebihan alur dalam pembuatan sistem nantinya. Pada penelitian ini terdapat tiga tahapan perancangan yang meliputi perancangan perangkat keras (*hardware*), perancangan perangkat lunak (*software*) dan perancangan antarmuka (*interface*) sistem.

3.4.1 *Hardware*

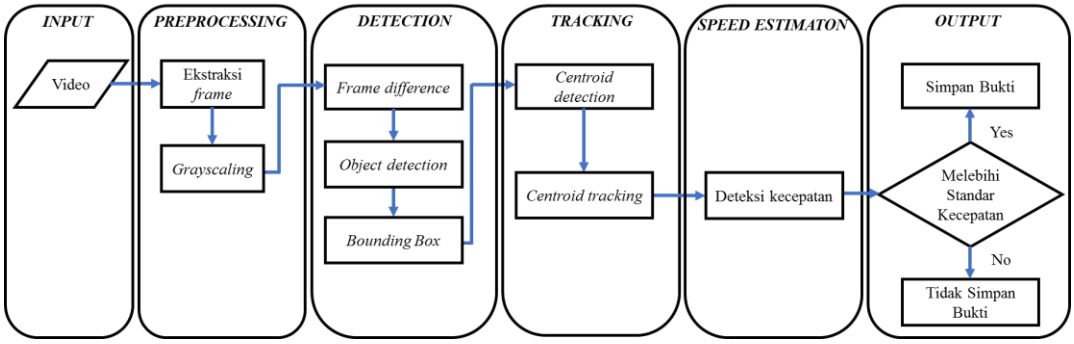
Hardware pada penelitian ini dirancang menggunakan dua kompunen yaitu satu unit Raspberry pi 3 model B dan satu unit USB *camera* logitech C310 dengan resolusi 5MP. Dua komponen tersebut akan disatukan menjadi satu kesatuan *device* yang akan digunakan untuk keseluruhan proses dalam sistem mulai dari perekaman video hingga menyimpan bukti pelanggaran. *Device* hasil dari penyatuan dua komponen tersebut dapat dilihat pada Gambar.3.4.



Gambar 3.4 *Device* yang terdiri dari Raspberry pi 3 dan USB *camera* Logitech

3.4.2 *Software*

Software pendeteksi kecepatan kendaraan ini dibangun menggunakan bahasa pemrograman Python versi 3. Proses pembangunan *software* dilakukan di dalam perangkat Raspberry pi 3 model B. Alur kerja sistem dapat dilihat pada Gambar 3.5. Video yang menjadi *input* akan diambil *frame*-nya mulai dari *frame* pertama hingga *frame* terakhir. *Frame* yang didapat dari video akan dikonversi menjadi citra *grayscale*. Setelah itu setiap *frame* yang sudah disiapkan akan diolah menggunakan metode *frame difference* untuk mendeteksi ada atau tidaknya objek bergerak pada setiap *frame*. Setiap objek yang terdeteksi akan diberi kotak penanda (*bounding box*). Berikutnya titik tengah dari setiap *bounding box* (*centroid*) akan dideteksi koordinatnya sehingga pergerakan dari *centroid* tersebut dapat ditelusuri koordinat awal dan koordinat akhirnya (*centroid tracking*). Hasil dari *centroid tracking* nantinya akan digunakan sebagai acuan untuk menghitung kecepatan kendaraan yang terdeteksi. Terakhir sistem akan menentukan apakah kecepatan objek yang terdeteksi melebihi kecepatan standar atau tidak. Jika kecepatan yang terdeteksi melebihi kecepatan standar berkendara yang telah ditentukan maka bukti pelanggaran berupa foto akan disimpan.



Gambar 3.5 Alur kerja sistem

3.4.2.1 *Input*

Input merupakan sebuah variabel yang akan diolah dalam sebuah sistem. Sistem pendeteksi kecepatan ini dibangun untuk mengolah masukan berupa video hasil rekaman dan bukan video *realtime*. Hal tersebut dikarenakan kemampuan perangkat dalam mengolah video *realtime* memberikan hasil yang kurang

maksimal. Saat perangkat dijalankan untuk menerima dan mengolah masukan secara *realtime* terjadi jeda yang cukup lama antara pergerakan objek secara *realtime* dengan pergerakan objek yang tertangkap kamera. Keterlambatan perangkat dalam menangkap objek secara *realtime* tentunya berpengaruh terhadap sistem dalam mendeteksi pergerakan objek. Sebagai ilustrasi dapat dilihat perbedaan posisi objek secara *realtime* pada Gambar 3.6 dengan posisi objek yang tertangkap oleh perangkat pada Gambar 3.7.

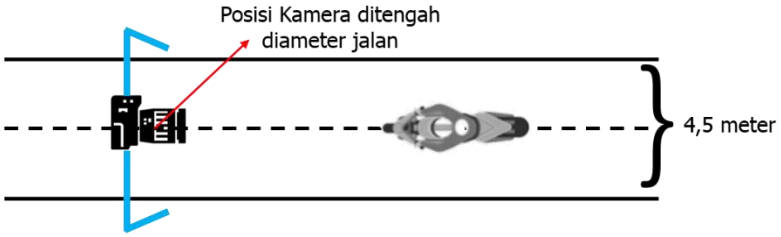


Gambar 3.6 Posisi objek secara realtime

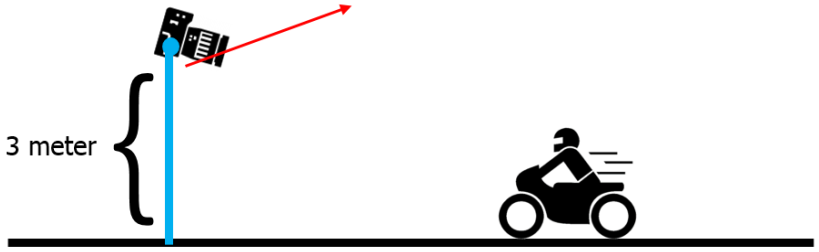


Gambar 3.7 Posisi objek hasil penangkapan kamera

Dari Gambar 3.6 dan Gambar 3.7 dapat dilihat bahwa jeda yang dihasilkan antara objek *realtime* dengan objek yang ditangkap oleh perangkat cukup besar. Unt uk mengatasi kekurangan tersebut maka sistem dirancang untuk mengolah video dari hasil rekaman. Video diambil dari lokasi penelitian menggunakan kamera digital Logitech C310 dengan resolusi 5MP. Kamera yang sudah disatukan dengan perangkat *Raspberry Pi 3 model B* sehingga menjadi satu kesatuan *device* akan diposisikan sebagai kamera statis dengan sudut kemiringan sebesar 60° terhadap jalan. Ilustrasi penempatan kamera terhadap jalan (tampak atas) dapat dilihat pada Gambar 3.8 sedangkan Ilustrasi penempatan kamera terhadap jalan (tampak samping) dapat dilihat pada Gambar 3.9.



Gambar 3.8 Ilustrasi penempatan kamera terhadap jalan (tampak atas)

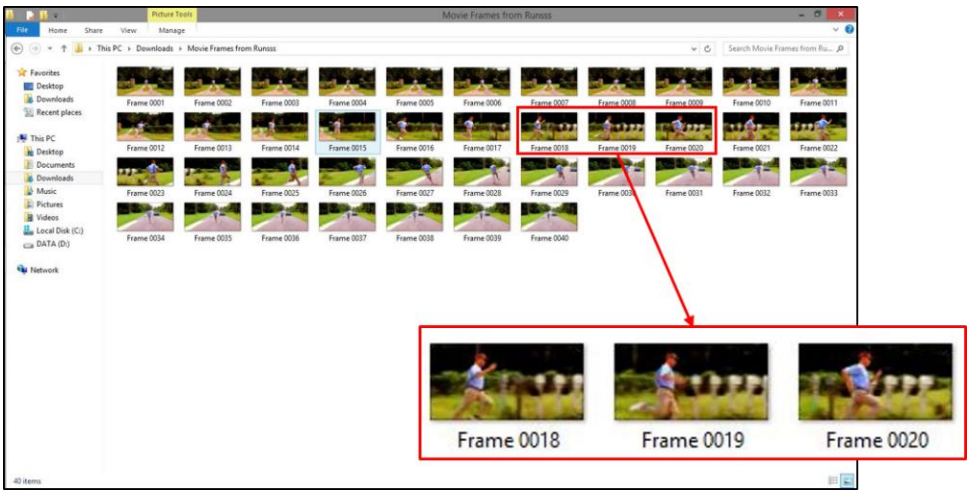


Gambar 3.9 Ilustrasi penempatan kamera terhadap jalan (tampak samping)

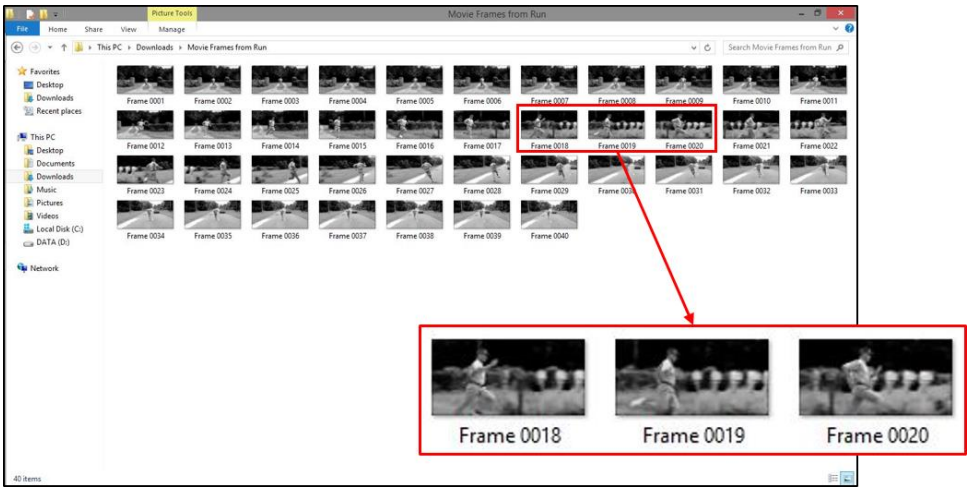
Pada proses pengambilan video, ada beberapa variabel yang perlu diperhatikan, yaitu intensitas cahaya, kecepatan pada *speedometer*, ketinggian posisi kamera dan sudut kemiringan kamera terhadap jalan.

3.4.2.2 *Preprocessing*

Pada tahap ini seluruh *frame* yang ada pada video akan diekstrak. Setelah diekstrak, *frame_n* yang diasumsikan sebagai *frame* sekarang (contoh: *frame_1*) dan *frame_n-1* yang diasumsikan sebagai satu *frame* sebelumnya (contoh: *frame_0*) akan dipanggil oleh sistem dan dikonversi menjadi citra *grayscale* agar dapat dideteksi selisih nilai pikselnya menggunakan metode *frame difference* pada tahap berikutnya. Untuk lebih jelasnya mengenai ilustrasi hasil ekstraksi *frame* dari video dapat dilihat pada Gambar 3.10 dan ilustrasi hasil konversi citra *frame* RGB menjadi citra *frame grayscale* dapat dilihat pada Gambar 3.11.



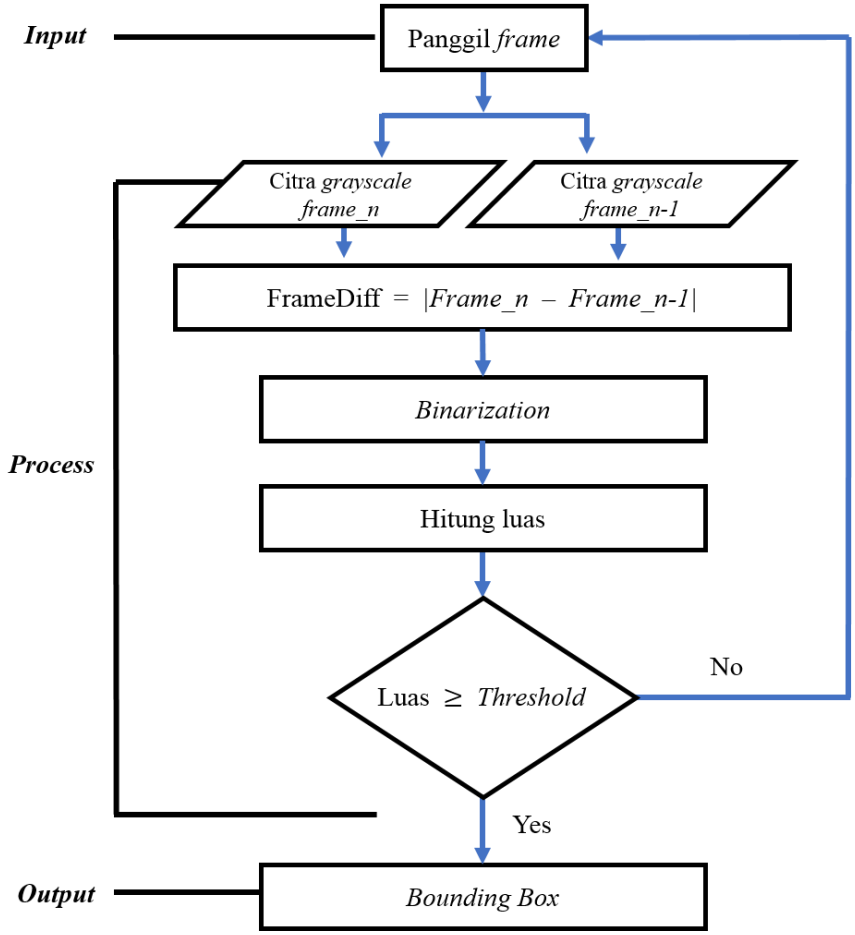
Gambar 3.10 Hasil ekstraksi frame dari sebuah video orang berlari



Gambar 3.11 Hasil konversi citra frame RGB menjadi citra frame grayscale

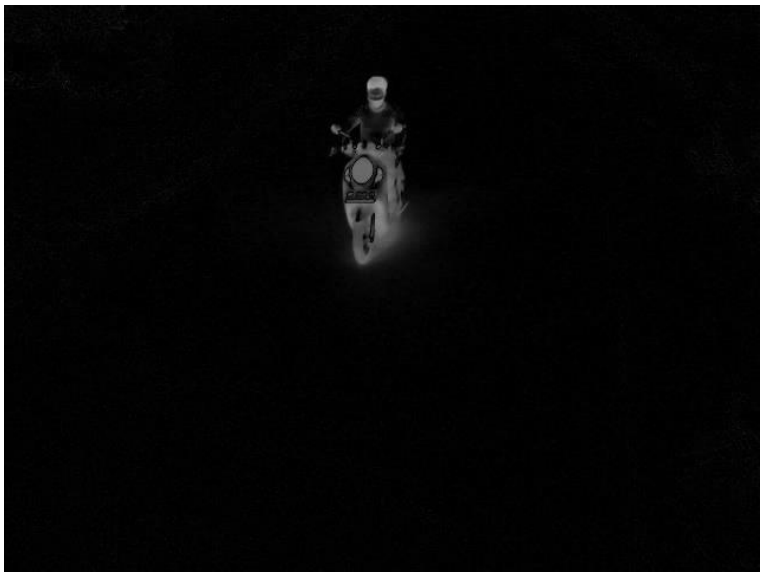
3.4.2.3 *Detection*

Setelah tahap *preprocessing* berhasil dilakukan, maka tahap berikutnya adalah mendeteksi adanya objek bergerak pada video dengan metode *frame difference*. Pada penelitian ini objek bergerak yang dideteksi adalah sepeda motor. Alur kerja metode dalam mendeteksi sepeda motor pada video dapat dilihat pada Gambar 3.12.



Gambar 3.12 Alur kerja metode frame difference untuk deteksi objek

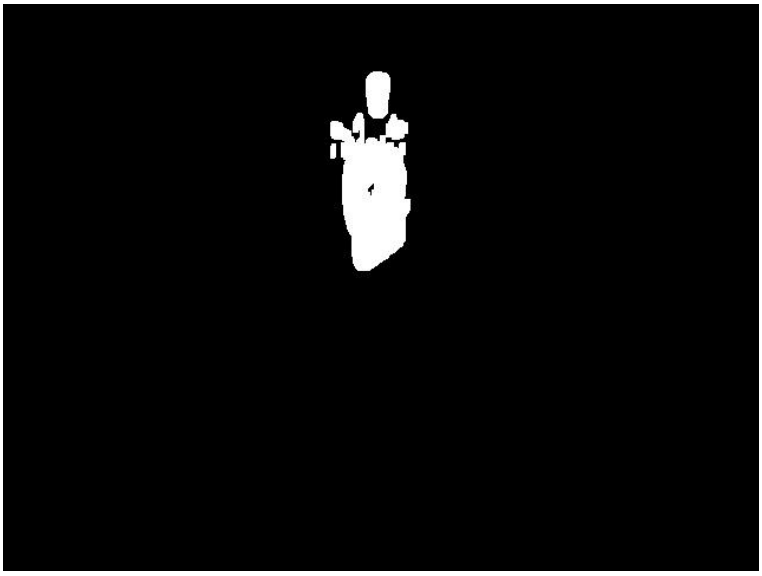
Bagan di atas menunjukkan bahwa pemanggilan citra *grayscale* dari hasil ektraksi *frame* merupakan *input* yang akan diproses dalam mendeteksi objek. *frame_n* yang diasumsikan sebagai *frame* sekarang akan dicari nilai differensialnya dengan *frame_n-1* yang merupakan satu *frame* sebelumnya. Ilustrasi hasil differensial dapat dilihat pada gambar 3.13.



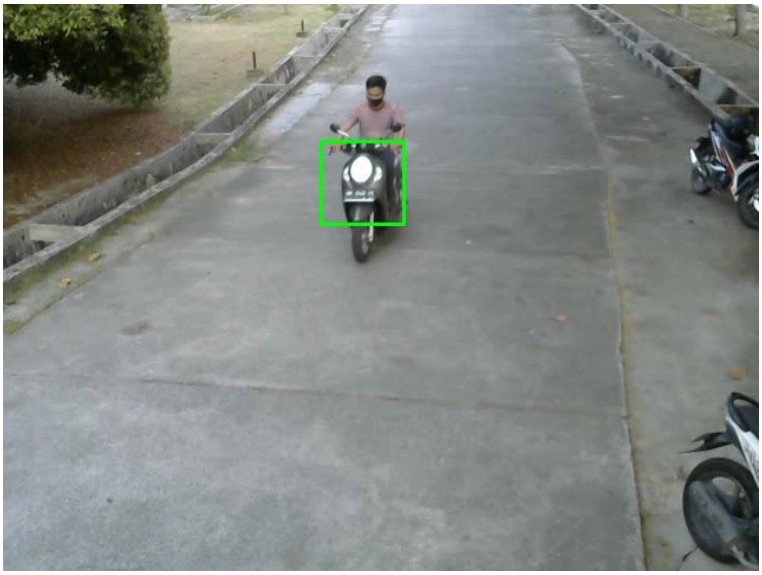
Gambar 3.13 Ilustrasi gambar hasil frame difference

Berikutnya gambar hasil differensial tersebut akan diubah menjadi citra biner (*binarization*) seperti yang diperlihatkan pada Gambar 3.14. Proses *binarization* dilakukan agar luas dari objek yang terdeteksi dapat dihitung. Luas yang didapatkan akan dibandingkan dengan nilai *threshold*.

Threshold yang digunakan merupakan nilai rata-rata luas kendaraan bermotor pada video yang terdeteksi pada penelitian ini. Jika nilai luas objek yang terdeteksi besar dari atau sama dengan nilai *threshold* maka objek tersebut akan diasumsikan sebagai sepeda motor. Hal tersebut perlu dilakukan agar objek-objek yang terdeteksi selain sepeda motor (*noise*) dapat diasumsikan sebagai objek lain dan bukan sepeda motor. Jika objek yang terdeteksi merupakan sepeda motor, maka objek akan diberikan kotak penanda (*bounding box*). Ilustrasi objek dengan *bounding box* pada suatu *frame* dapat dilihat pada Gambar 3.15.



Gambar 3.14 Binarisasi hasil dari frame difference

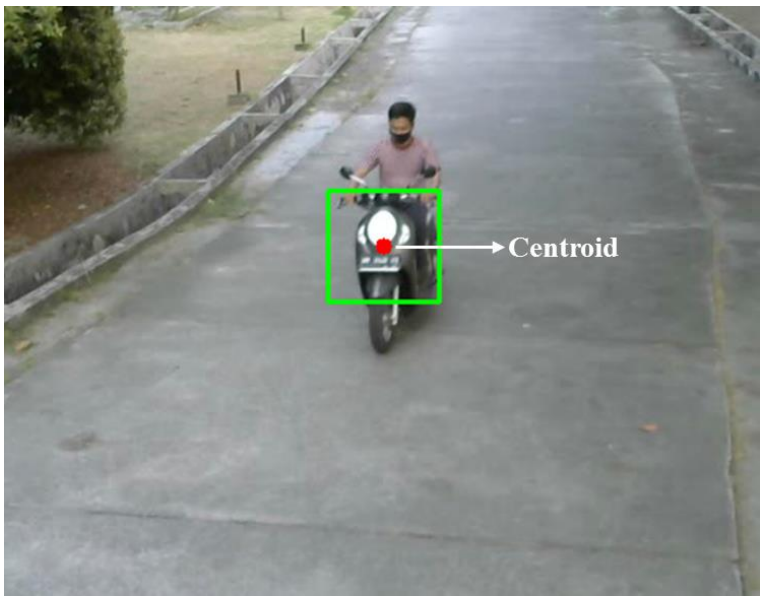


Gambar 3.15 Pemberian bounding box pada kendaraan terdeteksi

Pada Gambar 3.15. dapat dilihat bahwa *bounding box* merupakan sebuah persegi dengan empat sudut dan sisi. Setiap sudut dari *bounding box* memiliki koordinat (x,y) yang menandakan posisinya pada *frame*. Koordinat yang didapatkan dari pemberian *bounding box* digunakan untuk keperluan *centroid detection* yang akan dilakukan pada tahap berikutnya.

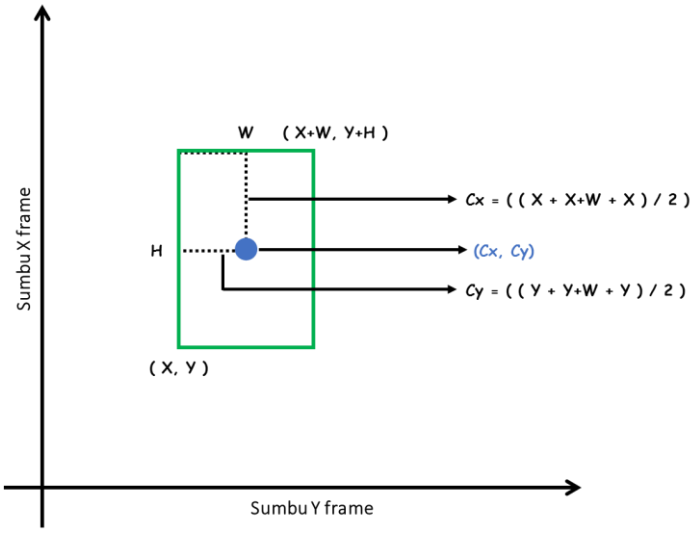
3.4.2.4 *Tracking*

Setelah kendaraan bermotor terdeteksi maka langkah berikutnya adalah mencari jalur pergerakan kendaraan di dalam video tersebut. Tahapan yang diperlukan untuk mendapatkan jalur pergerakan kendaraan adalah *centroid detection* dan *centroid tracking*. *Centroid* merupakan titik tengah dari suatu objek pada sebuah gambar. Ilustrasi posisi *centroid* dari suatu objek yang terdeteksi pada gambar dapat dilihat pada Gambar 3.16.



Gambar 3.16 Centroid dari suatu objek yang terdeteksi pada gambar

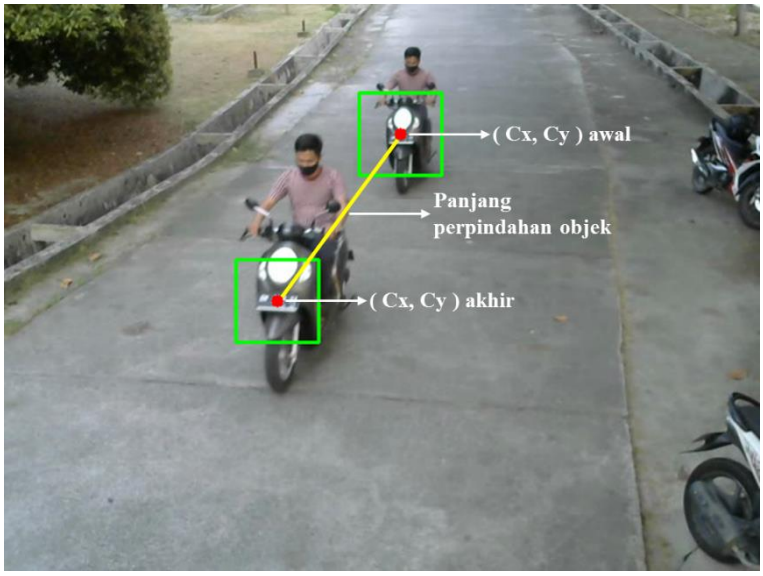
Centroid detection merupakan sebuah langkah untuk mendeteksi koordinat *centroid* dari suatu objek pada gambar. Pendeteksian koordinat *centroid* dapat dilakukan dengan menghitung besaran setengah dari lebar dan panjang *bounding box*. Hasil dari setengah nilai lebar dan setengah nilai panjang *bounding box* merupakan koordinat dari *centroid*. Ilustrasinya ditunjukkan pada Gambar 3.17.



Gambar 3.17 Perhitungan koordinat centroid

Dari Gambar 3.17 dapat dilihat bahwa (Cx, Cy) merupakan koordinat dari *centroid* yang ingin dideteksi. Cx merupakan koordinat *centroid* pada sumbu X, sedangkan Cy merupakan koordinat *centroid* pada sumbu Y. Hasil dari pendeteksian posisi *centroid* ini akan dijadikan acuan dalam melakukan *centroid tracking*.

Centroid tracking merupakan tahapan untuk mengetahui perpindahan posisi kendaraan pada video. Perpindahan kendaraan dapat diketahui dengan cara menyimpan (Cx,Cy) dari posisi awal kendaraan terdeteksi hingga posisi akhir kendaraan terdeteksi. Ilustrasi hasil dari *centroid tracking* yang memperlihatkan posisi awal dan posisi akhir kendaraan dapat dilihat pada Gambar 3.18. Hasil dari *centroid tracking* ini nantinya akan digunakan sebagai acuan dalam menentukan jarak perpindahan kendaraan pada video.



Gambar 3.18 Hasil centroid tracking dari suatu kendaraan pada video

3.4.2.5 *Speed Estimation*

Setelah melalui beberapa tahapan di atas, maka berikutnya sistem akan melakukan perhitungan perkiraan kecepatan (*speed estimation*) kendaraan yang terdeteksi. Pada umumnya rumus perhitungan kecepatan melibatkan dua variabel yaitu variabel jarak dan variabel waktu. Dengan adanya dua variabel tersebut maka kecepatan dari sebuah objek bergerak dapat kita hitung dengan rumus sebagai berikut.

$$Kecepatan = \frac{Jarak}{Waktu} \tag{3.1}$$

Rumus perhitungan kecepatan di atas hanya berlaku jika jarak yang terlibat dalam perhitungan memiliki satuan meter dan waktu yang terlibat memiliki satuan detik. Hal tersebut menyebabkan perhitungan kecepatan objek pada sebuah video yang memiliki satuan jarak piksel dan satuan waktu *frame per second* tidak dapat dihitung secara langsung. Oleh sebab itu penyesuaian satuan jarak dan waktu dari objek pada video perlu dilakukan dengan cara menghitung jarak *euclidean* terlebih dahulu menggunakan persamaan 3.2.

$$Jarak\ Euclidean = \sqrt{(Cx_1 - Cx_2)^2 + (Cy_1 - Cy_2)^2} \quad (3.2)$$

Keterangan:

Cx_1 = koordinat *centroid* awal pada sumbu x

Cx_2 = koordinat *centroid* aakhir pada sumbu x

Cy_1 = koordinat *centroid* awal pada sumbu y

Cy_1 = koordinat *centroid* akhir pada sumbu y

Setelah jarak *euclidean* didapatkan maka kecepatan dari kendaraan yang sudah terdeteksi dapat dihitung menggunakan persamaan 3.3.

$$Deteksi\ Kecepatan = \frac{Je \times FPS \times Jr}{\frac{1}{FPS}\ x\ N} \quad (3.3)$$

Keterangan:

Jr = Jarak referensi atau nilai 1 piksel dalam meter.

FPS = Jumlah *frame per second*

Je = Jarak *euclidean*.

N = Jumlah *frame* yang dibutuhkan oleh objek untuk melewati garis acuan.

Hasil dari perhitungan kecepatan akan digunakan sebagai referensi untuk menentukan apakah pengendara melebihi kecepatan standar yang telah ditentukan atau tidak.

3.4.2.6 *Output*

Output merupakan keluaran atau hasil akhir dari suatu sistem yang telah dibuat. Hasil akhir dari sistem pendeteksi kecepatan ini merupakan gambar bukti pelanggaran batas kecepatan di area kampus Universitas Riau. Di dalam gambar bukti pelanggaran tersebut terdapat foto nyata kendaraan yang melanggar serta detail informasi pelanggaran yang meliputi lokasi pelanggaran, batas standar

kecepatan yang diperbolehkan dan kecepatan kendaraan yang terdeteksi melanggar. Ilustrasi gambar bukti pelanggaran yang dihasilkan oleh sistem pada penelitian ini dapat dilihat pada Gambar 3.19.



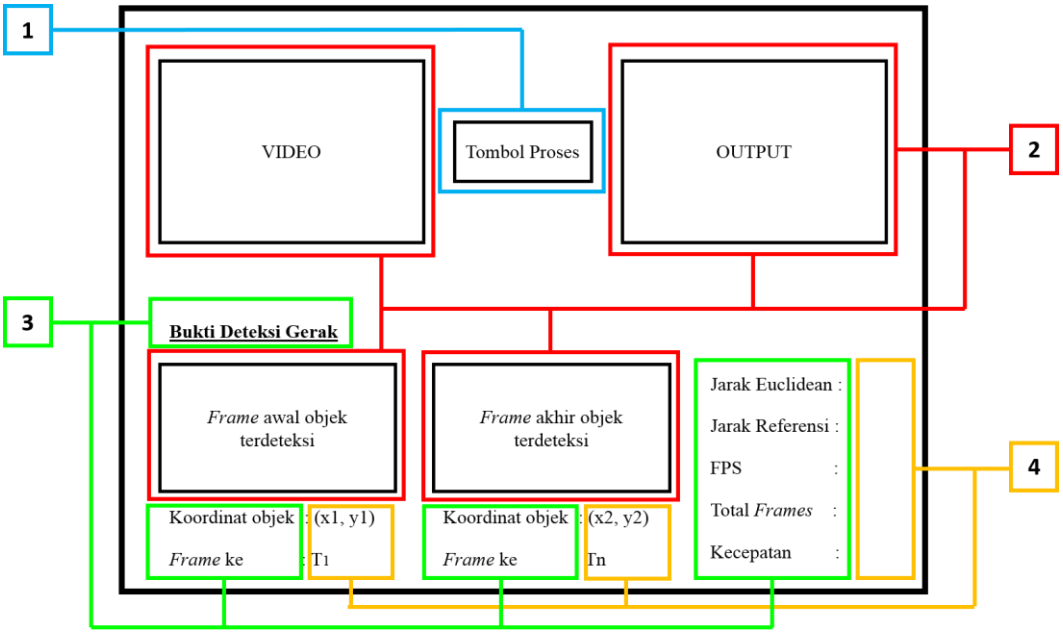
Gambar 3.19 Bukti pelanggaran batas standar kecepatan yang dihasilkan oleh sistem

Dari gambar di atas dapat kita lihat bahwa gambar utama merupakan gambar motor yang melanggar. Sedangkan *watermark* yang ditambahkan adalah detail informasi pelanggaran yang dilakukan oleh kendaraan dan diposisikan di bagian bawah gambar utama. Setiap detail informasi diberikan label dengan warna yang berbeda agar lebih mudah dipahami dan lebih menarik secara visual.

Pemberian *watermark* pada gambar dilakukan dengan bantuan salah satu *library* dari *Python* yaitu *Pillow*. *Pillow package* menyediakan beberapa *tools* seperti *image* yang digunakan untuk membaca gambar, *ImageDraw* untuk membuat gambar yang dibaca menjadi *drawable* atau dapat diedit dan *ImageFont* untuk menentukan format *font* atau jenis huruf yang dapat digunakan pada sebuah gambar baik itu *watermark* berupa teks ataupun gambar (Mike, 2017).

3.4.3 *Interface*

Antarmuka dibuat untuk menampilkan informasi terkait sistem kepada pengguna dan memberi kemungkinan kepada pengguna untuk berinteraksi dengan sistem. Untuk memaksimalkan penyajian informasi serta interaksi pengguna dengan sistem, maka desain antarmuka perlu dirancang seefisien mungkin. Desain antarmuka pada sistem pendeteksian kecepatan kendaraan ini dirancang dengan menggunakan *Graphical User Interface* (GUI) pada *python* yaitu *Tkinter python*. Visualisasi rancangan desain antarmuka dapat dilihat pada Gambar 3.20.



Gambar 3.20 Desain Antarmuka Sistem

Tabel 3.1 Keterangan Gambar 3.20

No	Operator	Fungsi
1	*Button*	Memulai sistem mulai dari *input* hingga menghasilkan *output*
2	*Picture Label*	Menampilkan *preview* video dan *output*
3	*Text*	Memberikan keterangan informasi keluaran dari sistem
4	*Text Label*	Menampilkan hasil keluaran berupa *text*

Dari desain antarmuka pada bagian ”VIDEO” akan menampilkan *preview* dari *input* video. Sedangkan pada bagian “*OUTPUT*” akan menampilkan hasil keluaran sistem. Informasi “Bukti Deteksi Gerak” juga ditampilkan yang mencakup *preview frame* awal dan *frame* akhir dideteksinya objek serta informasi mengenai objek yang terdeteksi yang mencakup koordinat awal objek dan koordinat akhir objek. Informasi mengenai elemen-elemen yang digunakan untuk acuan penghitungan kecepatan juga ditampilkan yang mencakup jarak Euclidean, jarak referensi, FPS, total *frame* dan hasil dari pendeteksian kecepatan itu sendiri.

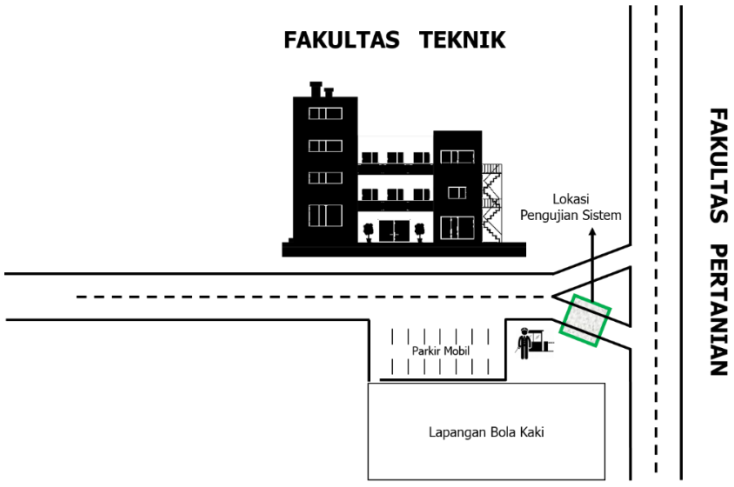
3.5 Pembuatan Sistem

Pada tahap ini sistem yang telah dirancang akan direalisasikan sesuai dengan alur yang telah dibuat. Sistem dibangun di dalam perangkat Raspberry pi 3 model b. Pembuatan sistem dimulai dengan implementasi hasil rancangan desain antarmuka menggunakan Tkinter *python*. Berikutya *coding program* akan ditulis menggunakan bahasa pemrograman *python* 3. Penulisan *code program* diurut sesuai dengan alur yang telah ditentukan.

3.6 Pengujian Sistem

3.6.1 Lokasi Pengujian Sistem

Pengujian sistem akan dilakukan di jalan masuk ke kawasan fakultas teknik Universitas Riau. Untuk lebih jelasnya dapat dilihat pada Gambar 3.21.



Gambar 3.21 Detail lokasi pengujian sistem

3.6.2 Tahap Pengujian Sistem

Pengujian pada sistem pendeteksi kecepatan kendaraan ini akan dilakukan dalam beberapa keadaan. Pertama sistem akan diujikan pada keadaan pagi hari dengan intensitas cahaya 3000 - 4000 lux untuk mendeteksi 10 unit kendaraan bermotor dengan 3 kecepatan berbeda per unitnya yaitu 30km/jam, 40km/jam dan 45km/jam. berikutnya sistem akan diujikan pada keadaan sore hari dengan intensitas cahaya antara 1.500 – 1.600 lux untuk mendeteksi 10 unit kendaraan bermotor dengan 3 kecepatan berbeda per unitnya yaitu 30km/jam, 40km/jam dan 45km/jam.

Tahapan pengujian yang akan dilakukan pada sistem baik pada keadaan pagi hari atau sore hari adalah sebagai berikut.

1. Menyiapkan 10 unit kendaraan bermotor yang akan dijadikan objek uji coba.
2. Memasang alat pendeteksi kecepatan yang sudah dibuat pada area pengujian.
3. Mengendarai motor melewati area perekaman kamera dengan 3 kecepatan yang sudah ditentukan secara berurutan mulai dari motor 1 hingga motor 10.
4. Hasil perekaman akan diproses oleh sistem agar dapat dideteksi kecepatan dari motor yang ada pada video.
5. Hasil deteksi kecepatan oleh sistem akan dibandingkan dengan kecepatan asli motor pada *speedometer* untuk mengetahui tingkat keakuratan sistem dalam mendeteksi kecepatan motor.

3.7 Analisis Hasil

Analisis hasil merupakan tahapan penyajian data hasil dari pengujian sistem. Data hasil pengujian disajikan dalam bentuk tabel yang berisi informasi lengkap mengenai hasil pengujian mulai dari kendaraan bermotor pertama hingga kendaraan bermotor terakhir dengan kecepatan mulai dari 30km/jam, 40km/jam dan 45km/jam dalam keadaan pagi dan sore hari.

Setelah disajikan, data hasil uji coba sistem akan dianalisis guna mengetahui tingkat kemampuan sistem dalam mendeteksi kecepatan. Berikutnya akurasi sistem dalam mendeteksi status pelanggaran akan dihitung menggunakan metode

confusion matrix. Akurasi didapatkan dengan bantuan tabel *multiclass confusion matrix* yang ditunjukkan pada Tabel 3.2.

Tabel 3.2 *Multiclass confusion matrix*

n		Prediksi	
		Melanggar	Tidak Melanggar
Aktual	Melanggar	TM	FM
	Tidak Melanggar	FTM	TTM

Dari Tabel 3.2 terdapat ”n” yang merupakan total jumlah hasil deteksi, “Aktual” yang menandakan status pelanggaran asli, “Prediksi” yang menandakan hasil prediksi sistem dalam menentukan status pelanggaran, “TM” atau *true* melanggar yang merupakan jumlah dari hasil deteksi “Melanggar” dan status aslinya juga “Melanggar”, “FM” atau *false* melanggar merupakan jumlah dari hasil deteksi “Tidak Melanggar” namun status aslinya “Melanggar”, “FTM” atau *false* tidak melanggar merupakan jumlah hasil deteksi “Melanggar” namun status aslinya “Tidak Melanggar”, terakhir ada “TTM” atau *true* tidak melanggar yang merupakan jumlah hasil deteksi “Tidak Melanggar” dan status aslinya juga “Tidak Melanggar”.

Perhitungan akurasi dapat dilakukan apabila nilai dari TM, FM, FTM dan TTM sudah ada. Berikutnya setiap nilai yang ada akan dihitung menggunakan rumus yang diperlihatkan pada Persamaan 3.6.

$$Akurasi = \frac{TM + TTM}{TM + FM + TTM + FTM} \tag{3.4}$$

Keterangan:

TM = *true* melanggar

FM = *false* melanggar

TTM = *true* tidak melanggar

FTM = *false* tidak melanggar

3.8 Kesimpulan

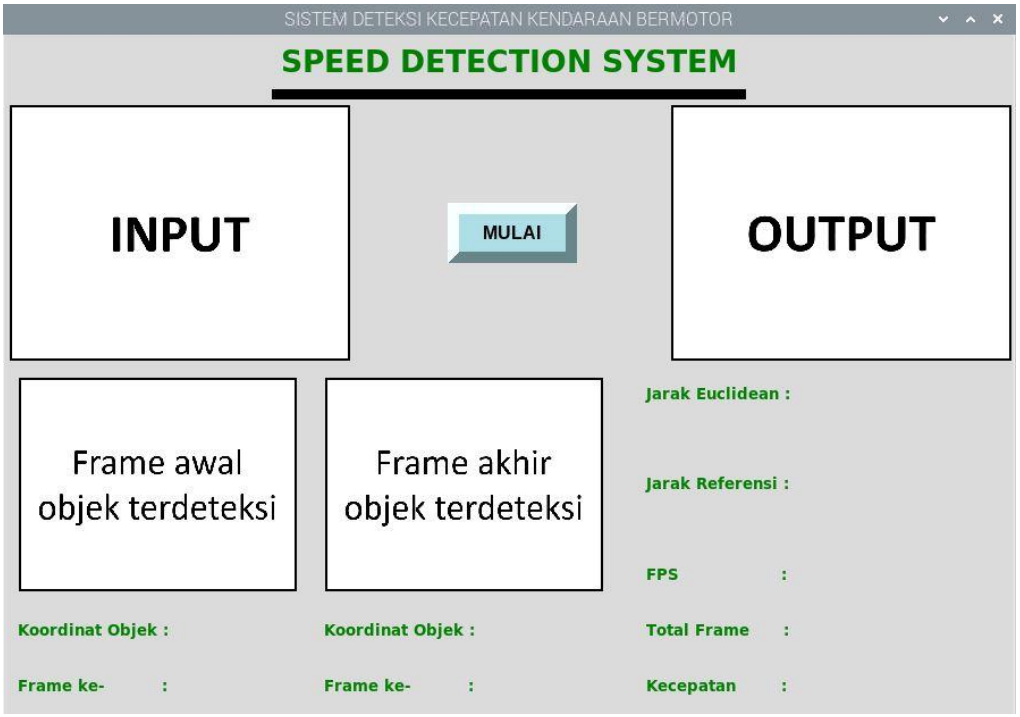
Penarikan kesimpulan dilakukan dengan mengacu pada studi pustaka dan rumusan permasalahan serta hasil analisis dari penelitian. Simpulan yang akan diperoleh merupakan jawaban dari bagaimana cara mendeteksi kendaraan bermotor bergerak pada sebuah video yang ditangkap oleh kamera digital dan bagaimana cara menghitung kecepatan pergerakan kendaraan bermotor pada sebuah video yang ditangkap oleh kamera digital serta rangkuman mengenai hasil terbaik dari anaslisis pengujian sistem.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil

4.1.1 Hasil Pembuatan Sistem

Hasil pembuatan sistem akan ditampilkan dalam bentuk gambar yang merupakan *screeenshoot* dari sistem yang telah dibuat. Hasil *screenshoot* GUI sistem dapat dilihat pada Gambar 4.1.



Gambar 4.1 Screenshoot GUI sistem

Dari Gambar 4.1 dapat dilihat bahwa hanya diberikan satu *button* yaitu "MULAI". Di dalam *button* tersebut terdapat keseluruhan *coding* sistem pendeteksi kecepatan mulai dari membaca masukan hingga menghasilkan *output*. Proses yang dijalankan dalam *button* tersebut adalah :

1. Membaca masukan

   Berikut kode untuk proses membaca video masukan

```
vidcap = cv2.VideoCapture('video_file_directory_and_name')
```

2. Memecah video menjadi *frame*

   Berikut kode ekstraksi *frame* dari video

```
def getFrame(sec):
    vidcap.set(cv2.CAP_PROP_POS_MSEC,sec*1000)
    hasFrames,image = vidcap.read()
    if hasFrames:
        cv2.imwrite(pathOut+"image"+str(count)+".jpg", image)
    return hasFrames
```

3. *Grayscaling*

   Sebelum proses *grayscaling* dilakukan, dilakukan pemanggilan *frame* hasil ekstraksi dari video. Setelah itu setiap 2 *frame* akan dipanggil secara berurutan untuk diubah menjadi citra *grayscale*. Berikut kode yang digunakan untuk tahapan proses *grayscaling*

```
for i in col_frames:
    img = cv2.imread('frames/frames'+ind_speed+'/'+i)
    blk = np.zeros(img.shape, np.uint8)
    image = Image.open('frames/frames'+ind_speed+'/'+i)
    col_images.append(img)
    col_images_forcrop.append(image)
for i in range(len(col_images)-1):
    #grayscaling
    grayA = cv2.cvtColor(col_images[i], cv2.COLOR_BGR2GRAY)
    grayB = cv2.cvtColor(col_images[i+1], cv2.COLOR_BGR2GRAY)
```

4. *Frame difference* untuk mengenali objek

   Berikut kode yang digunakan untuk proses *frame difference*

```
for i in range(len(col_images)-1):
    #grayscaling
    grayA = cv2.cvtColor(col_images[i], cv2.COLOR_BGR2GRAY)
    grayB = cv2.cvtColor(col_images[i+1], cv2.COLOR_BGR2GRAY)
    #frame differencing
    diff_image = cv2.absdiff(grayB, grayA)
    # image thresholding
    ret, thresh = cv2.threshold(diff_image, 50, 255, cv2.THRESH_BINARY)
    # image dilation
    dilated = cv2.dilate(thresh,kernel,iterations = 3)
    # find contours
    cnts = cv2.findContours (dilated.copy(), cv2.RETR_LIST, cv2.CHAIN
    _APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)
```

5. *Centroid detection*

   Berikut kode untuk proses deteksi *centroid*

```
for cntr in cnts:
        (x, y, w, h) = cv2.boundingRect(cntr)
        #centroid_coor
        cx = int((x + x + w) / 2.0)
        cy = int((y + y + h) / 2.0)
```

6. *Centroid tracking*

Centroid yang sudah ditemukan koordinatnya akan dilacak pergerakannya menggunakan kode berikut

```
if (x <= 640) & (cy >= 110) & (cv2.contourArea(cntr) >= 4000):
        if (cy >= 300):
            break
        luas = (w*h)
        if (luas) >= 2000:
            if cy <= 120:
                frame_counter = frame_counter+1
                Centro_1 = (cx,cy)
                Cnt_1_1st = cx
                Cnt_1_2nd = cy
                bukti_pelanggaran = col_images[i].copy()
            if cy > 120:
                frame_counter = frame_counter+1
                Centro_2 = (cx,cy)
                Cnt_2_1st = cx
                Cnt_2_2nd = cy
            #Drawing_Bbox
            cv2.rectangle(dmy, (cx - 35, cy - 35), (cx + 35, cy + 35), (0, 255,
            0), 2)
            #Drawing_Centroid
            cv2.circle(dmy, (cx, cy), 5, (0, 0, 255), -1)
            cv2.putText(dmy, "centroid", (cx + 5, cy),cv2.FONT_HERSHEY
            _SIMPLEX, 0.5, (0, 0, 0), 2)
```

7. *Speed calculation*

Tahapan perhitungan kecepatan diperlihatkan pada kode berikut

```
#jarak perpindahan objek pada video
    d_pixels = math.sqrt(math.pow(Cnt_2_1st - Cnt_1_1st, 2) + math.pow
    (Cnt_2_2nd - Cnt_1_2nd, 2))
    # frame yang terpakai dibagi fps
    fpf = (frame_counter/10)
    d_meters = d_pixels/fpf
    #print("d_pixels=" + str(d_pixels), "d_meters=" + str(d_meters))
    fps = 10
    speed = int((d_meters * fps * 8.7)/1000)
    print ("Speed : " + str(speed))
```

8. Menentukan status pelanggaran dan menyimpan bukti pelanggaran

Kecepatan yang didapatkan akan dijadikan acuan untuk menentukan status pelanggaran dari kendaraan tersebut. Berikut kode yang digunakan untuk proses penentuan status pelanggaran dan menyimpan bukti pelanggaran

```
if (speed!=0) & (speed > 40):
    # Draw Watermark
    cv2.rectangle(bukti_pelanggaran, (10, 400), (180, 450), (0, 255, 255), -1)
    cv2.putText(bukti_pelanggaran, "Lokasi: F.Teknik Unri", (15, 430), font
    , 0.5, (0, 0, 0), 2)
    cv2.rectangle(bukti_pelanggaran, (210, 400), (430, 450), (0, 255, 0), -1)
    cv2.putText(bukti_pelanggaran, "Batas Kecepatan: 40km/j", (215, 430),
    font, 0.5, (0, 0, 0), 2)
    cv2.rectangle(bukti_pelanggaran, (460, 400), (630, 450), (0, 0, 255), -1)
    cv2.putText(bukti_pelanggaran, "Kecepatan: "+str((speed))+"km/j",
    (465, 430), font, 0.5, (0, 0, 0), 2)
```

```
# Specify the text location and rotation angle
text_location = (110, 300)
angle = 20

# Draw the text using cv2.putText()
cv2.rectangle(blk, (110-10, 300-70), (110+465, 300+24), (0, 0, 255), 3)
cv2.putText(blk, 'MELANGGAR', text_location, font, 2.5, (0, 0, 255), 5)

# Rotate the image using cv2.warpAffine()
M = cv2.getRotationMatrix2D(text_location, angle, 1)
editedtext = cv2.warpAffine(blk, M, (blk.shape[1], blk.shape[0]))

# Generate result by blending both images
out = cv2.addWeighted(bukti_pelanggaran, 1.0, editedtext, 1.0, 1)
cv2.imwrite("bukti_pelanggaran"+ind_speed+".jpg", out)
```

4.1.2 Hasil Pengujian Sistem

Pada bagian ini hasil dari pengujian sistem akan ditampilkan. Hasil pengujian yang dimaksud mencakup pengujian deteksi deteksi kecepatan kendaraan pada video di setiap kondisi yang telah ditentukan. Data akan ditampilkan dalam bentuk tabel. Hasil pengujian deteksi kecepatan pada *speedometer* 30 km/j dapat dilihat pada Tabel 4.1. Untuk *speedometer* 40 km/j dapat dilihat pada Tabel 4.2. Dan untuk *speedometer* 45 km/j dapat dilihat pada Tabel 4.3.

Tabel 4.1 Data hasil pengujian deteksi deteksi kecepatan pada *speedometer* 30 km/j

Waktu	no	kendaraan	speedometer	deteksi kecepatan	deteksi pelanggaran	
					Asli	Sistem
PAGI	1	Kendaraan 1	30	29	TIDAK	TIDAK
	2	Kendaraan 2	30	31	TIDAK	TIDAK
	3	Kendaraan 3	30	29	TIDAK	TIDAK
	4	Kendaraan 4	30	31	TIDAK	TIDAK
	5	Kendaraan 5	30	30	TIDAK	TIDAK
	6	Kendaraan 6	30	28	TIDAK	TIDAK
	7	Kendaraan 7	30	32	TIDAK	TIDAK
	8	Kendaraan 8	30	32	TIDAK	TIDAK
	9	Kendaraan 9	30	28	TIDAK	TIDAK
	10	Kendaraan 10	30	30	TIDAK	TIDAK
SORE	11	Kendaraan 1	30	30	TIDAK	TIDAK
	12	Kendaraan 2	30	29	TIDAK	TIDAK
	13	Kendaraan 3	30	31	TIDAK	TIDAK
	14	Kendaraan 4	30	26	TIDAK	TIDAK
	15	Kendaraan 5	30	32	TIDAK	TIDAK
	16	Kendaraan 6	30	26	TIDAK	TIDAK
	17	Kendaraan 7	30	32	TIDAK	TIDAK
	18	Kendaraan 8	30	27	TIDAK	TIDAK
	19	Kendaraan 9	30	31	TIDAK	TIDAK
	20	Kendaraan 10	30	32	TIDAK	TIDAK

Tabel 4.2 Data hasil pengujian deteksi deteksi kecepatan pada *speedometer* 40 km/j

Waktu	no	kendaraan	speedometer	deteksi kecepatan	deteksi pelanggaran	
					Asli	Sistem
PAGI	1	Kendaraan 1	40	40	TIDAK	TIDAK
	2	Kendaraan 2	40	39	TIDAK	TIDAK
	3	Kendaraan 3	40	38	TIDAK	TIDAK
	4	Kendaraan 4	40	42	TIDAK	YA
	5	Kendaraan 5	40	39	TIDAK	TIDAK
	6	Kendaraan 6	40	40	TIDAK	TIDAK
	7	Kendaraan 7	40	41	TIDAK	YA
	8	Kendaraan 8	40	38	TIDAK	TIDAK
	9	Kendaraan 9	40	37	TIDAK	TIDAK
	10	Kendaraan 10	40	39	TIDAK	TIDAK

Waktu	no	kendaraan	speedometer	deteksi kecepatan	deteksi pelanggaran	
					Asli	Sistem
SORE	11	Kendaraan 1	40	38	TIDAK	TIDAK
	12	Kendaraan 2	40	40	TIDAK	TIDAK
	13	Kendaraan 3	40	42	TIDAK	YA
	14	Kendaraan 4	40	38	TIDAK	TIDAK
	15	Kendaraan 5	40	38	TIDAK	TIDAK
	16	Kendaraan 6	40	39	TIDAK	TIDAK
	17	Kendaraan 7	40	42	TIDAK	YA
	18	Kendaraan 8	40	40	TIDAK	TIDAK
	19	Kendaraan 9	40	42	TIDAK	YA
	20	Kendaraan 10	40	40	TIDAK	TIDAK

Tabel 4.3 Data hasil pengujian deteksi deteksi kecepatan pada speedometer 45 km/j

Waktu	no	kendaraan	speedometer	deteksi kecepatan	deteksi pelanggaran	
					Asli	Sistem
PAGI	1	Kendaraan 1	45	46	YA	YA
	2	Kendaraan 2	45	44	YA	YA
	3	Kendaraan 3	45	45	YA	YA
	4	Kendaraan 4	45	44	YA	YA
	5	Kendaraan 5	45	47	YA	YA
	6	Kendaraan 6	45	47	YA	YA
	7	Kendaraan 7	45	45	YA	YA
	8	Kendaraan 8	45	43	YA	YA
	9	Kendaraan 9	45	43	YA	YA
	10	Kendaraan 10	45	47	YA	YA
SORE	11	Kendaraan 1	45	47	YA	YA
	12	Kendaraan 2	45	45	YA	YA
	13	Kendaraan 3	45	43	YA	YA
	14	Kendaraan 4	45	46	YA	YA
	15	Kendaraan 5	45	44	YA	YA
	16	Kendaraan 6	45	40	YA	TIDAK
	17	Kendaraan 7	45	44	YA	YA
	18	Kendaraan 8	45	43	YA	YA
	19	Kendaraan 9	45	48	YA	YA
	20	Kendaraan 10	45	47	YA	YA

4.2 Analisis Hasil

4.2.1 Analisis hasil deteksi kecepatan

Dari data yang disajikan pada Tabel 4.1, Tabel 4.2 dan Tabel 4.3 dapat dilihat kemampuan sistem dalam mendeteksi kecepatan. Untuk lebih jelasnya, kinerja sistem dalam mendeteksi kecepatan di keadaan pagi hari dijabarkan dalam Tabel 4.4. Untuk kinerja sistem pada sore hari dijabarkan pada Tabel 4.5. Dan kinerja sistem secara keseluruhan disajikan pada Tabel 4.6.

Tabel 4.4 Analisis hasil kinerja sistem pada keadaan pagi hari

Speedometer	**Hasil Deteksi**		
	Terbaik	**Terburuk**	**Rata-Rata**
30 km/j	30 km/j	28 km/j	30 km/j
40 km/j	40 km/j	37 km/j	39,3 km/j
45 km/j	45 km/j	43 km/j dan 47 km/j	45,1 km/j

Dari Tabel 4.4 pada *speedometer* 30 km/j dapat dilihat bahwa hasil deteksi terbaik adalah 30 km/j dengan selisih sebesar 0 km/j. Sedangkan hasil deteksi terburuk ada pada 28 km/j dengan selisih dengan *speedometer* sebesar 2 km/j. Rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 30 km/j adalah 30 km/j.

Pada *speedometer* 40 km/j, sistem dapat mendeteksi dengan hasil terbaik berada pada 40 km/j dengan selisih sebesar 0 km/j. Sedangkan untuk hasil deteksi terburuk berada pada 37 km/j dengan selisih sebesar 3 km/j. Untuk rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 40 km/j adalah 39,3 km/j.

Untuk *speedometer* 45 km/j, hasil deteksi kecepatan terbaik ada pada 45 km/j dengan selisih sebesar 0 km/j. Dan untuk hasil deteksi terburuk berada pada 43 km/j dan 47 km/j dengan selisih dengan *speedometer* sebesar 2 km/j. Sedangkan rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 45 km/j adalah 45,1 km/j.

Tabel 4.5 Analisis hasil kinerja sistem pada keadaan sore hari

Speedometer	**Hasil Deteksi**		
	Terbaik	**Terburuk**	**Rata-Rata**
30 km/j	30 km/j	26 km/j	29,6 km/j
40 km/j	40 km/j	38 km/j dan 42 km/j	39,9 km/j
45 km/j	45 km/j	40 km/j	44,7 km/j

Dari Tabel 4.4 pada *speedometer* 30 km/j dapat dilihat bahwa hasil deteksi terbaik adalah 30 km/j. Sedangkan hasil deteksi terburuk yang dihasilkan oleh sistem dalam mendeteksi kecepatan berada pada 26 km/j. Rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 30 km/j adalah 29,6 km/j.

Pada *speedometer* 40 km/j, sistem dapat mendeteksi dengan hasil terbaik yaitu 40 km/j. Sedangkan untuk hasil terburuk berada pada 38 km/j dan 42km/j. Untuk rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 40 km/j adalah 39,9 km/j.

Untuk *speedometer* 45 km/j, hasil deteksi kecepatan terbaik ada pada 45 km/j dengan selisih sebesar 0 km/j dengan *speedometer*. Dan untuk hasil deteksi terburuk dengan nilai selisih mencapai 5 km/j berada pada 40 km/j. Sedangkan rata-rata hasil deteksi kecepatan dari 10 kendaraan pada *speedometer* 45 km/j adalah 44,7 km/j.

Tabel 4.6 Analisis kinerja sistem secara keseluruhan

Speedometer	**Hasil Deteksi**		
	Terbaik	**Terburuk**	**Rata-Rata**
30 km/j	30 km/j	26 km/j	29,8 km/j
40 km/j	40 km/j	37 km/j	39,6 km/j
45 km/j	45 km/j	40 km/j	44,9 km/j

Dari Tabel 4.6 ditunjukkan hasil deteksi secara keseluruhan. Pada *speedometer* 30 km/j dapat dilihat bahwa hasil deteksi terbaik adalah 30 km/j. Sedangkan hasil deteksi teburuk yang dihasilkan oleh sistem dalam mendeteksi

kecepatan berada pada 26 km/j. Rata-rata hasil deteksi kecepatan dari 20 kendaraan pada *speedometer* 30 km/j adalah 29,8 km/j.

Pada *speedometer* 40 km/j, sistem dapat mendeteksi dengan hasil terbaik yaitu 40 km/j. Sedangkan untuk hasil terburuk berada pada 37 km/j. Untuk rata-rata hasil deteksi kecepatan dari 20 kendaraan pada *speedometer* 40 km/j adalah 39,6 km/j.

Untuk *speedometer* 45 km/j, hasil deteksi terbaik ada pada 45 km/j dan untuk hasil deteksi terburuk berada pada 40km/j. Sedangkan rata-rata hasil deteksi kecepatan dari 20 kendaraan pada *speedometer* 45 km/j adalah 44,9 km/j.

4.2.2 Analisis hasil deteksi pelanggaran

Selain data hasil deteksi kecepatan, kemampuan sistem dalam menentukan status pelanggaran dari setiap kecepatan yang dideteksi juga ditampilkan pada Tabel 4.1, Tabel 4.2 dan Tabel 4.3. Detail hasil pendeteksian status pelanggaran pada kecepatan 30 km/j ditampilkan pada Tabel 4.7, sedangkan untuk data pada kecepatan 40 km/j ditampilkan pada Tabel 4.9 dan data pada kecepatan 45 km/j ditampilkan pada Tabel 4.11.

Tabel 4.7 Hasil pendeteksian status pelanggaran pada kecepatan 30 km/j

Waktu	Speedometer	Status Asli Pelanggaran	Hasil Deteksi Sistem	
			Melanggar	Tidak Melanggar
PAGI	30 km/j	Tidak Melanggar	0	10
SORE	30 km/j	Tidak Melanggar	0	10

Akurasi sistem dalam mendeteksi status pelanggaran pada kecepatan 30 km/j dapat diketahui dengan membuat tabel *multiclass confusion matrix* terlebih dahulu. Data yang dijadikan acuan untuk membuat tabel *multiclass confusion matrix* merupakan data hasil deteksi pelanggaran yang disediakan pada Tabel 4.7.

Tabel 4.8 *Multiclass confusion matrix* pada kecepatan 30 km/j

n = 20		Prediksi	
		Melanggar	Tidak Melanggar
Aktual	Melanggar	TM = 0	FM = 0
	Tidak Melanggar	FTM = 0	TTM = 20

Berikutnya akurasi dapat dihitung menggunakan rumus akurasi *confusion matrix*:

$$Akurasi = \frac{TM + TTM}{TM + FM + TTM + FTM} = \frac{0 + 20}{0 + 0 + 20 + 0} = \frac{20}{20} = 1$$

$$Akurasi = 1\ x\ \frac{100}{100} = 100\%$$

Tabel 4.9 Hasil pendeteksian status pelanggaran pada kecepatan 40 km/j

Waktu	Speedometer	Status Asli Pelanggaran	Hasil Deteksi Sistem	
			Melanggar	Tidak Melanggar
PAGI	40 km/j	Tidak Melanggar	2	8
SORE	40 km/j	Tidak Melanggar	3	7

Akurasi sistem dalam mendeteksi status pelanggaran pada kecepatan 40 km/j dapat diketahui dengan membuat tabel *multiclass confusion matrix* terlebih dahulu. Data yang dijadikan acuan untuk membuat tabel *multiclass confusion matrix* merupakan data hasil deteksi pelanggaran yang disediakan pada Tabel 4.9.

Tabel 4.10 *Multiclass confusion matrix* pada kecepatan 40 km/j

n = 20		Prediksi	
		Melanggar	Tidak Melanggar
Aktual	Melanggar	TM = 0	FM = 0
	Tidak Melanggar	FTM = 5	TTM = 15

Berikutnya akurasi dapat dihitung menggunakan rumus akurasi *confusion matrix*:

$$Akurasi = \frac{TM + TTM}{TM + FM + TTM + FTM} = \frac{0 + 15}{0 + 0 + 15 + 5} = \frac{15}{20} = 0{,}75$$

$$Akurasi = 0{,}75\ x\ \frac{100}{100} = 75\%$$

Tabel 4.11 Hasil pendeteksian status pelanggaran pada kecepatan 45 km/j

Waktu	Speedometer	Status Asli Pelanggaran	Hasil Deteksi Sistem	
			Melanggar	Tidak Melanggar
PAGI	45 km/j	Melanggar	10	0
SORE	45 km/j	Melanggar	9	1

Akurasi sistem dalam mendeteksi status pelanggaran pada kecepatan 45 km/j dapat diketahui dengan membuat tabel *multiclass confusion matrix* terlebih dahulu. Data yang dijadikan acuan untuk membuat tabel *multiclass confusion matrix* merupakan data hasil deteksi pelanggaran yang disediakan pada Tabel 4.11.

Tabel 4.12 *Multiclass confusion matrix* pada kecepatan 45 km/j

n = 20		Prediksi	
		Melanggar	Tidak Melanggar
Aktual	Melanggar	TM = 19	FM = 1
	Tidak Melanggar	FTM = 0	TTM = 0

Berikutnya akurasi dapat dihitung menggunakan rumus akurasi *confusion matrix*:

$$Akurasi = \frac{TM + TTM}{TM + FM + TTM + FTM} = \frac{19 + 0}{19 + 1 + 0 + 0} = \frac{19}{20} = 0{,}95$$

$$Akurasi = 0{,}95\ x\ \frac{100}{100} = 95\%$$

Tabel 4.13 Hasil pendeteksian status pelanggaran secara keseluruhan

Speedometer	Status Asli Pelanggaran	Hasil Deteksi Sistem	
		Melanggar	Tidak Melanggar
30 km/j	Tidak Melanggar	0	20
40 km/j	Tidak Melanggar	5	15
45 km/j	Melanggar	19	1

Akurasi sistem dalam mendeteksi status pelanggaran secara keseluruhan pada tiga kategori kecepatan 30 km/j, 40 km,j dan 45 km/j dapat diketahui dengan membuat tabel *multiclass confusion matrix* terlebih dahulu. Data yang dijadikan acuan untuk membuat tabel *multiclass confusion matrix* merupakan data hasil deteksi pelanggaran yang disediakan pada Tabel 4.13.

Tabel 4.14 *Multiclass confusion matrix* hasil deteksi keseluruhan

n = 60		Prediksi	
		Melanggar	Tidak Melanggar
Aktual	Melanggar	TM = 19	FM = 1
	Tidak Melanggar	FTM = 5	TTM = 35

Berikutnya akurasi dapat dihitung menggunakan rumus akurasi *confusion matrix*:

$$Akurasi = \frac{TM + TTM}{TM + FM + TTM + FTM} = \frac{19 + 35}{19 + 1 + 35 + 5} = \frac{54}{60} = 0{,}90$$

$$Akurasi = 0{,}90\ x\ \frac{100}{100} = 90\%$$

Dari empat perhitungan yang sudah diselesaikan dapat dilihat bahwa pada kecepatan 30 km/j sistem mampu mencapai akurasi sebesar 100%, pada kecepatan 40 km/j sistem mampu mencapai akurasi 75%, pada kecepatan 45 km/j sistem mampu mencapai akurasi sebesar 95%, dan secara keseluruhan sistem dapat mencapai akurasi sebesar 90% dalam mendeteksi status pelanggaran.

BAB V
PENUTUP

5.1 Kesimpulan

Setelah penelitian ini selesai dilakukan, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Perangkat Raspberry pi 3 model B mampu menjalankan protoype sistem pendeteksi kecepatan yang dibangun menggunakan bahasa pemrograman Python 3 dalam penelitian ini.
2. Metode *frame difference* memberikan hasil yang cukup baik dalam mendeteksi objek pada video masukan yang ditentukan pada penelitian ini. Pendeteksian objek yang berhasil dicapai pada penelitian ini dipengaruhi oleh beberapa faktor yang meliputi intensitas cahaya antara 1.500 – 4.000 Lux, posisi kamera ditempatkan di tengah diameter jalan dengan tinggi 3 meter dan sudut kemiringan terhadap jalan sebesar 60 derajat serta kecepatan pergerakan objek antara 30 – 45 km/j dengan acuan *speedometer* yang ada pada kendaraan.
3. Dari analisis hasil dapat dilihat bahwa pada kecepatan 30 km/j sistem mampu mencapai akurasi sebesar 100%, pada kecepatan 40 km/j sistem mampu mencapai akurasi 75%, pada kecepatan 45 km/j sistem mampu mencapai akurasi sebesar 95%, dan secara keseluruhan sistem dapat mencapai akurasi sebesar 90% dalam mendeteksi status pelanggaran.

5.2 Saran

Beberapa saran pengembangan dapat dilakukan terhadap penelitian ini, yaitu:

1. Peningkatan perangkat dari raspberry pi 3 ke raspberry 4 untuk meningkatkan performa sistem dapat dilakukan.
2. Mengingat sistem yang telah dibuat ini hanya dapat mengenali *single object*, maka peningkatan untuk pengenalan *multiple object* dapat dilakukan.
3. Peningkatan dari hanya memproses video hasil rekaman ke sistem *realtime* dapat dilakukan.
4. Penambahan variasi sudut kamera terhadap jalan dapat dilakukan.
5. Penambahan variasi intensitas cahaya untuk pengujian sistem dapat dilakukan.
6. Penambahan variasi objek yang dideteksi kecepatannya juga dpat dilakukan.

DAFTAR PUSTAKA

Alex, D., & Wahi, A. (2014). *BSFD : BACKGROUND SUBTRACTION FRAME DIFFERENCE ALGORITHM FOR MOVING OBJECT*. *60*(3), 623–628.

Cilsy. (2018). *Tutorial Raspberry Pi 3*. https://www.cilsy.id/

Gonzales, Woods, & Eddins. (2004). *Digital Image Processing Using Matlab. Prentice Hall*. https://www.scribd.com/document/176249519/

Gupta, P., Singh, Y., & Gupta, M. (2014). *Moving Object Detection Using Frame Difference , Background Subtraction And SOBS For Video Surveillance Application*. 151–156.

Ilham. (2019). *Pengertian Python, Fungsi, Kelebihan dan Kekurangan*. https://materibelajar.co.id/pengertian-python/

IWD, I. W. D. (2019). *Kelebihan dan Kekurangan Bahasa Pemrograman Python*. https://webdev-id.com/news/kelebihan-dan-kekurangan-bahasa-pemrograman-python

Marieski, T. C., Hidayat, B., & Safitri, I. (2017). *PERANCANGAN APLIKASI DETEKSI KEMACETAN BERDASARKAN PENGOLAHAN VIDEO DIGITAL MENGGUNAKAN METODE FRAME DIFFERENCE BERBASIS ANDROID Traffic Detection Application Design By Video Digital Proccessing Using Frame Difference Method Based On Android Keywords : tr*. *4*(1), 480–491.

Mike. (2017). *How to Watermark Your Photos with Python*.

Muhamad, R., Yulianti, T., Sulistiyanti, S. R., Purwiyanti, S., & Setyawan, F. X. A. (2019). *Deteksi Objek Bergerak Pada Video Bawah Air Menggunakan Metode Frame Differencing*. *13*(2), 100–104.

Nuralim, F., Irawan, B., & Prasasti, A. L. (2019). *Analisis dan Deteksi Gerakan*

Kepanikan menggunakan Metode Frame Difference pada Sistem Monitoring Rumah. *6*(2), 5517–5524.

Perhubungan, K. (2015). *Peraturan Mentri Perhubungan Republik Indonesia Nomor PM 111 Tahun 2015 Tentang Cara Penetapan Batas Kecepatan*. https://www.ditjenpp.kemenkumham.go.id/,

Rohman, M. (n.d.). *Analisa Gerakan Manusia Pada Video Digital*.

Sadewo, S. S., Sumiharto, R., & Candradewi, I. (2015). *Sistem Pengukur Kecepatan Kendaraan Berbasis Pengolahan Video*. *5*(2), 177–186.

Singla, N. (2014). *Motion Detection Based on Frame Difference Method*. *4*(15), 1559–1565.

Zhang, Y., Wang, X., & Qu, B. (2012). *Three-Frame Difference Algorithm Research Based on Mathematical Morphology*. *29*, 2705–2709. https://doi.org/10.1016/j.proeng.2012.01.376